Gestão de Projeto de Software 2023/2024

Licenciatura em Engenharia Informática

João Cunha (jcunha@isec.pt)
Mickaël Santos (a21270946@isec.pt)

©2011-2023 João Cunha, DEIS-ISEC

1

Software Project Management 2023/2024

João Cunha (jcunha@isec.pt)
Mickaël Santos (a21270946@isec.pt)
Ricardo Filipe (ricardo.angelo.filipe@gmail.com)

©2011-2023 João Cunha, DEIS-ISEC

2

Outline
What's this course about?

"Software Project Management"

João Cunha, ISEC (jcunha@isec.pt)

©2011-2023 João Cunha, DEIS-ISEC

3



4

Objectives

- Software management is concerned with knowledge about the planning, organization, monitoring and control of all software life-cycle phases.
- Management is critical to ensure that:
  - software development projects are appropriate to an organization,
  - the work in different organizational units is coordinated,
  - software versions and configurations are maintained,
  - resources are available, when necessary,
  - the quality is as expected,
  - project work is divided appropriately,
  - communication is facilitated, and
  - progress is accurately charted.
- The main objective of this course is to provide knowledge, understanding and practice on software project management

©2011-2023 João Cunha, DEIS-ISEC

5

Learning Outcomes

- Upon completing this course, students should be able to:
  - Describe software development as an engineering discipline, using appropriate terminology.
  - Develop a software project in a team, following defined processes and good software engineering practices.
  - Plan a software development project using estimates, task scheduling, and resource allocation.
  - Apply best practices in quality management, risk management, team management, and stakeholder management.
  - Manage a software project by monitoring and controlling its progress, ensuring software delivery within the planned timeframes and costs, and managing stakeholder expectations.
  - Demonstrate knowledge of software development processes and methodologies, as well as the principles and fundamentals of software process improvement.

©2011-2023 João Cunha, DEIS-ISEC

6

Course Philosophy

- The course shall run around the development of a software project.
  - Following an **AGILE** methodology
- Theoretical classes shall be used for presentation and discussion of some of the subjects and study materials.
- Labs shall be used to apply the best practices in software engineering
- Labs shall also be used for software development, progress meetings with the instructor, demonstrations to the class, and clarifications about deliverables and tools.
- Students shall be assessed by their work throughout the semester.

©2011-2023 João Cunha, DEIS-ISEC

7

Topics

1. Traditional and Agile Software Development Lifecycle
   - Scrum
   - Waterfall
2. Software Project Planning
   - Creating a software development plan
   - Software requirements, use cases, user stories
   - Delivery planning
   - Iteration planning
   - Project estimation and scheduling
   - Architecture
3. Agile Software Project Management
   - Requirements management
   - Risk management
   - Team management
   - Iteration review
4. Quality Management
   - Quality plan
   - Reviews
   - Testing and test plans
   - Version control
5. Agile Software Development
   - Scrum ceremonies, team, artifacts
   - DevOps principles

©2011-2023 João Cunha, DEIS-ISEC

8

Project outline

- The goal of the project is to build a software product within the expected scope, cost, quality and deadline.
- However, before the project begins, it must be carefully planned.
- This project shall follow a defined project management plan and quality assurance plan.
- The teams shall have 4 or 5 members.
- The project shall be **finished** by the end of the semester, i.e. **last week of December 2023**.

  No extension!
- A final **presentation** will take plane the **first week of January 2024**

©2011-2023 João Cunha, DEIS-ISEC

Grading policy

- Project grades are individual, and reflect both team performance and student individual contributions
- Final individual grades are based on:
  - Team grades
  - Peer assessment
  - Quality of produced artifacts (have your name on what you do)
  - Participation (on progress meetings, forums, etc.)
  - Initiative (don't wait for someone to tell you what to do)
  - Subjective perception of the instructor on the effective student achievements, sustained by self and peer assessment
- Students receive continuous feedback on their progress
- There is no project resubmission
- Final grade = Project (12) + Exam (8)
  - Minimum 40% each part

©2011-2023 João Cunha, DEIS-ISEC

Support

- Staring point: **inforestudante.ipc.pt**
- Platform: **teams.microsoft.com**
- Some support: **moodle.isec.pt**
- Development Platform: **GitLab**
- Office hours (Office or **MSTeams**)
- Communication between instructor and students, and team discussions: Posts or Chat at **MSTeams**
- Continuous feedback to students through **MSTeams**
- Individual Team Channel in **MSTeams**

©2011-2023 João Cunha, DEIS-ISEC

Student Working Hours

- Total: 160 hours (6 ECTS)
- Regular work, per week
  - 2 hours class
  - 3 hours lab (may be less)
  - **4 hours out-of-class work**
- Non-regular work
  - 23 hours study
  - 2 hours exam

©2011-2023 João Cunha, DEIS-ISEC

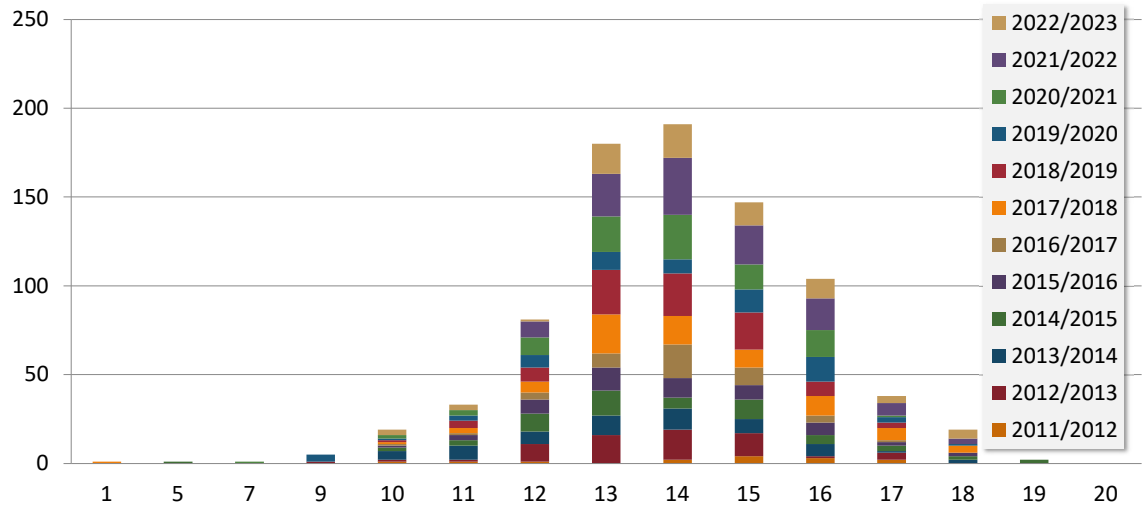
Bibliography

- Stellman, A., & Greene, J. (2006). Applied software project management. O'Reilly. 1A-7-73 (ISEC)
- Sommerville, I. (2011), Software Engineering, 9th ed. Pearson Education. 1A-7-107 (ISEC)
- Project Management Institute, Inc, A guide to the project management body of knowledge (PMBOK Guide) and the standard for project management, Newtown Square, ISBN/ISSN: ISBN 978-1-62825- 664-2, 2A-4-82 (ISEC)
- Ken Schwaber & Jeff Sutherland, The Scrum Guide - The Definitive Guide to Scrum: The Rules of the Game, November 2020 (https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf; https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Portuguese-European.pdf)
- James Shore and Shane Warden, The art of agile development, O'Reilly, cop. 2008, ISBN/ISSN: ISBN 978-0-596-52767-9, 1A-7-78 (ISEC)

©2011-2023 João Cunha, DEIS-ISEC

13

Grades 2011-2023



©2011-2023 João Cunha, DEIS-ISEC

14

Project Assignment

"Software Project Management"
João Cunha, ISEC (jcunha@isec.pt)
Mickaël Santos (a21270946@isec.pt

©2011-2023 João Cunha, DEIS-ISEC

15

1. Goal

- The goal of this project is to build a software product following a well-defined development plan and quality assurance plan, within the expected scope, quality, budget and deadline.

©2011-2023 João Cunha, DEIS-ISEC

16

2. Theme

- The product to be built shall be simple enough that a single person could program it in a couple of days.
- It should be original work.
- It should be built in java language.
- The choice of the theme is up to the team; however, it must be approved by the teacher.

©2011-2023 João Cunha, DEIS-ISEC

3. Teams

- Team size shall be, ideally, of 5 members (can be 4).
- All team members must be proficient in the Java programming language.
- Each member has a specific role:
  - Scrum Master (SM) - helps the Scrum Team understand and apply Scrum principles, ensuring effective collaboration and removing impediments to maximize the team's productivity and success.
  - Product Owner (PO) - is the individual responsible for defining and prioritizing the requirements, ensuring that the development team works on the most valuable features and aligns with the overall product vision and stakeholder needs in Agile development.
  - Agile Developers (Dev) - are all the team members who actively participate in Agile development methodologies.
- The roles change every sprin

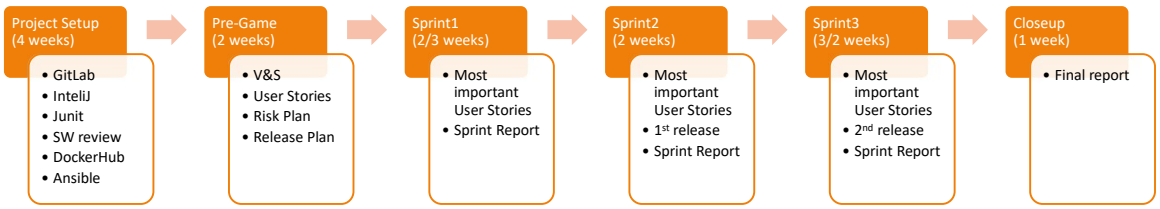
©2011-2023 João Cunha, DEIS-ISEC

4. Project repository

- Each team shall maintain a GitLab cloud repository to manage
  - Documents
  - Backlogs
  - Code
  - Sprints
  - Project status
  - Logs
  - …

©2011-2023 João Cunha, DEIS-ISEC

5. Project Lifecycle



©2011-2023 João Cunha, DEIS-ISEC

6. Work effort

- 4 *hours per person-week* (out of class)
- 4 x number of team members *hours per team-week* (ex: 4x5=20 *hours per week-team*)
- 4 x number of team members *hours per sprint* (ex: 4x5x2=40 or 4x5x3=60 *hours per sprint*)

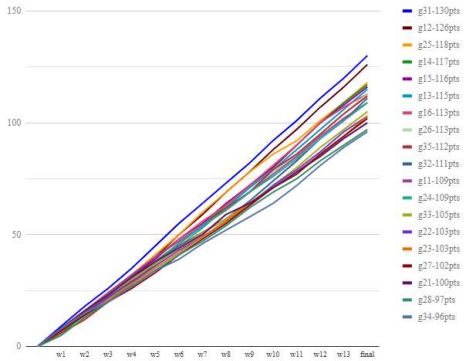
- The effort must be logged

©2011-2023 João Cunha, DEIS-ISEC

7. Project assessment

- From week 5, every week a progress meeting takes place
  - The meeting must be properly prepared
- The team members get feedback about the work performed

- The team is evaluated every week (scale 1-10).
  - Scores are public and compared.
  - Teams compete between each other!



©2011-2023 João Cunha, DEIS-ISEC

8. Evaluation and team rules

- The **teams** are qualitatively and quantitatively evaluated every week – grades are public
- The **students** are qualitatively evaluated every week – only for teacher records
- Students are encouraged to **assess the teammates** in the middle of the semester
- **The team will be graded** at the end of the semester
  - Then the **team members** will distribute unevenly the **grade** between them, reflecting the **individual contributions** to the project
- If one team member has a repeatedly negative attitude during the project, impairing team performance, he may be "invited" to **leave** the team
  - The teacher has the last word

©2011-2023 João Cunha, DEIS-ISEC

23

Good luck with your software project!



©2011-2023 João Cunha, DEIS-ISEC

24

1. Introduction
Why do projects fail?

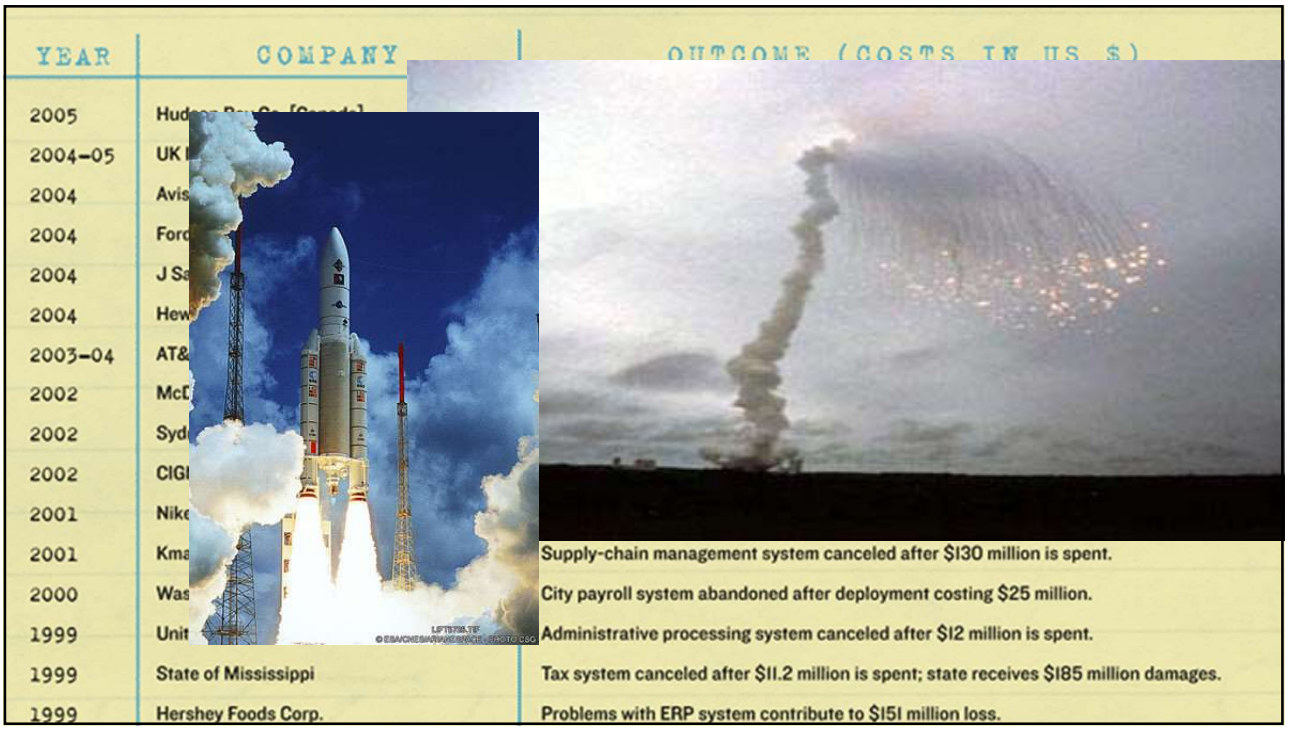
"Software Project Management"

João Cunha, ISEC (jcunha@isec.pt)

(Some slides from Andrew Stellman & Jennifer Greene, "Applied Software Project Management")

©2011-2023 João Cunha, DEIS-ISEC

25



26



27

The story of Vasa

- On 10 August 1628, the Royal Swedish Navy's newest ship set sail
- Vasa sailed about 1,300 meters and then, in a light gust of wind, capsized in Stockholm's harbor, losing 53 lives.
- The ship was Sweden's most expensive project ever.
- Sweden was at war with Poland and needed the ship for the war effort.
- A formal hearing the following month did not determine why the ship sank, and no one was blamed.
- The ship was removed from the waters, and a museum was built. The Vasa Museum is the most popular museum in Sweden – 25 million visitors since 1990

©2011-2023 João Cunha, DEIS-ISEC

28



29

What happened?

- King Gustav changed the shipbuilding orders
  - From 2 small and 2 big ships, to 4 larger ships, in reverse order, …
- No specifications for the modified kill
  - Change from 111-feet keel to all-new 135 feet
- Shift in armament requirements
  - Upgrade to two enclosed gun decks, with heavier guns
- The shipwright died
- No way to calculate stability, stiffness, or sailing characteristics
- The prelaunch stability test… failed

©2011-2023 João Cunha, DEIS-ISEC

30

Vasa… or software project problems?

1. Excessive schedule pressure
2. Changing needs
3. Lack of technical specifications
4. Lack of a documented project plan
5. Excessive and secondary innovations
6. Unclear requirements
7. Lack of scientific methods
8. Defects discovered too late
9. Unethical behavior

©2011-2023 João Cunha, DEIS-ISEC

Why do software projects fail?

- People begin programming before they understand the problem
  - When the team starts to code as soon as the project begins, they see immediate gains
  - When problems become more complex (as they always do!), the work gets bogged down
  - In the best case, a team that begins programming too soon will end up writing good software that solves the wrong problem
- The team has an unrealistic idea about how much work is involved.
  - From far away, most complex problems seem simple to solve
  - Teams can commit to impossible deadlines by being overly optimistic and not thinking through the work
  - Few people realize the deadline is optimistic until it's blown

©2011-2023 João Cunha, DEIS-ISEC

Why do software projects fail?

- Defects are injected early but discovered late.
  - Projects can address the wrong needs; requirements can specify incorrect behavior
  - Design, architecture and code can be technically flawed; Test plans can miss functionality
  - The later these problems are found, the more likely they are to cause the project to fail
- Programmers have poor habits
  - Programmers don't have good control of their source code
  - Code written by one person is often difficult for another person to understand
  - Programmers don't test their code, which makes diagnosing and fixing bugs more expensive
  - The team does not have a good sense of the overall health of the project.

©2011-2023 João Cunha, DEIS-ISEC

How can we make sure that our projects succeed?

- Introduce software quality from the very beginning of the project
  - Review everything, test everything.
  - Use reviews to find defects – but don't expect the review to be perfect.
  - Use reviews to gain a real commitment from the team.
  - It's always faster in the long run to hold a review than it is to skip it.
- Apply good engineering practices
  - The fastest way through the project is to use good engineering practices
  - Don't cut important tasks – especially estimation, reviews, requirements gathering and testing. If it were faster to build the software without these practices, we would never use them.
  - Every one of these practices is about saving time and increasing quality by planning well and finding defects early. Cutting them out will cost time and reduce quality.

©2011-2023 João Cunha, DEIS-ISEC

Let's start by… Identifying Needs

- The business manager drives the scope of the project.
  - He should identify and talk to the main stakeholder
  - The effective way to show stakeholders that their needs are understood and that those specific needs will be addressed is with a *vision and scope document*

©2011-2023 João Cunha, DEIS-ISEC

Vision and Scope Document

- A typical vision and scope document follows an outline like this one:

1. Problem Statement
   a) *Project background*
   b) *Stakeholders*
   c) *Users*
2. Vision of the Solution
   a) *Vision statement*
   b) *List of features*
   c) *Scope of phased release (optional)*
   d) *Features that will not be developed*
   e) *Risks*
   f) *Assumptions*

©2011-2023 João Cunha, DEIS-ISEC

Further reading

- Andrew Stellman and Jennifer Greene, *Applied Software Project Management*, O'Reilly, 2006. [Section 1 and 2]

©2011-2023 João Cunha, DEIS-ISEC

37

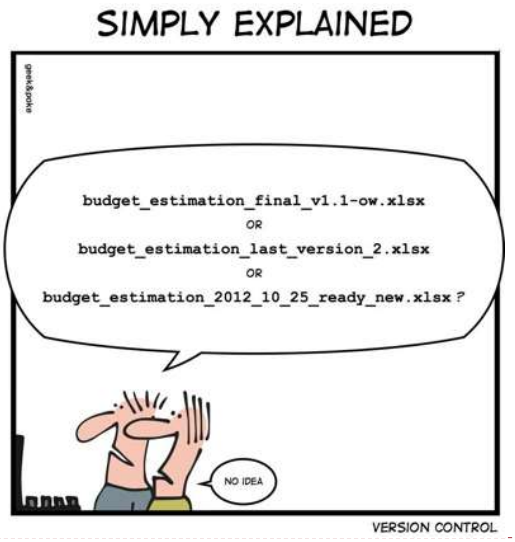
2. Software Version Control

"Software Project Management"

João Cunha, ISEC (jcunha@isec.pt)

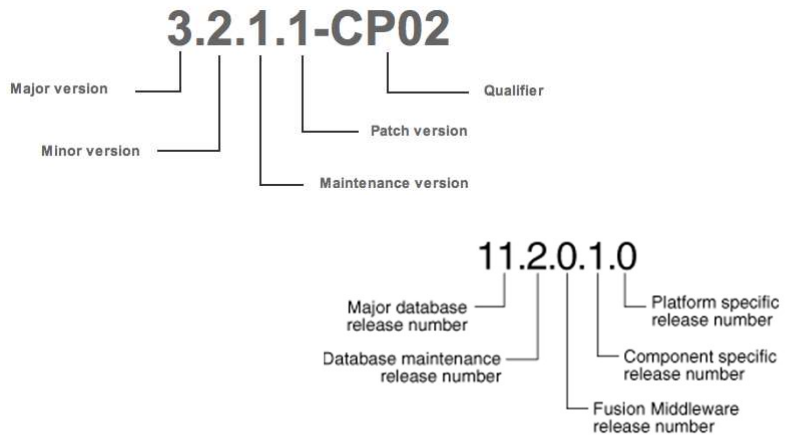
©2011-2023 João Cunha, DEIS-ISEC

38



©2011-2023 João Cunha, DEIS-ISEC

39



©2011-2023 João Cunha, DEIS-ISEC

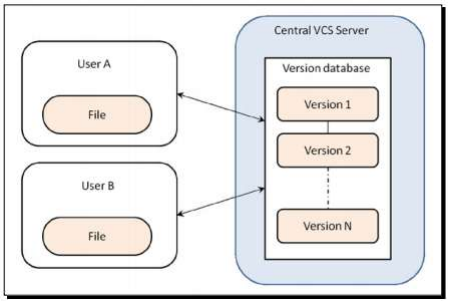
40

Configuration Management

- Configuration Management (CM) is the discipline that ensures that the state of the software at any given time is known and reconstructable. This refers both to software under development and released.
- **Version control** (or revision control, or source control) is the management of documents or programs, keeping track of all the changes.
- **Release control** manages the different releases of product lines, having a common set of files, and different build environments.

©2011-2023 João Cunha, DEIS-ISEC

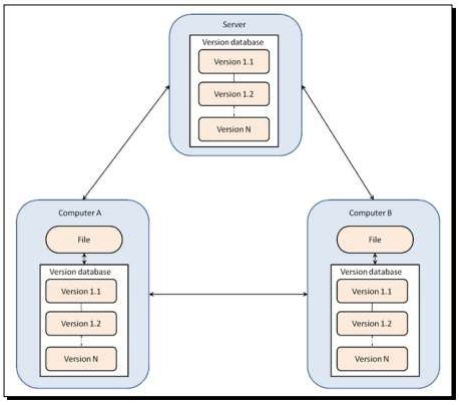
Centralized Version Control System



- Only the last version of the files are retrieved.
- Offers visibility on what other people are working towards.
- Any changes made to the files are automatically shared with other individuals as well.

©2011-2023 João Cunha, DEIS-ISEC

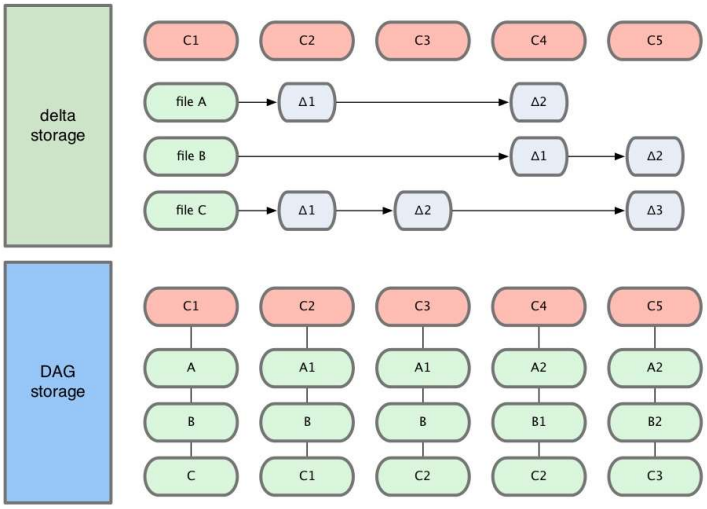
Distributed Version Control System



- Stores the entire history of the files on each and every machine locally
- Syncs the local changes made by the user back to the server whenever required so that the changes can be shared with others.
- Makes local changes without any concern of full-time connectivity to the server
- Does not rely on a single copy of files stored in the server

©2011-2023 João Cunha, DEIS-ISEC

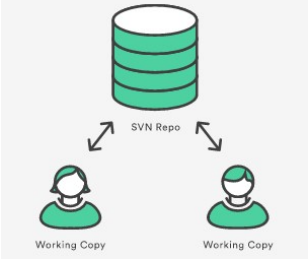
Storage sytem



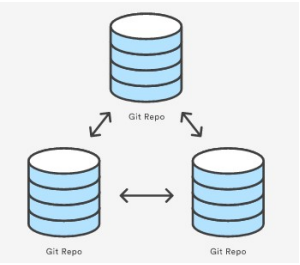
©2011-2023 João Cunha, DEIS-ISEC

SVN vs Git

- Apache Subversion (SVN)
  - Open-source project, founded in 2000 by CollabNet. Released in 2004
  - Centralized
  - Delta storage

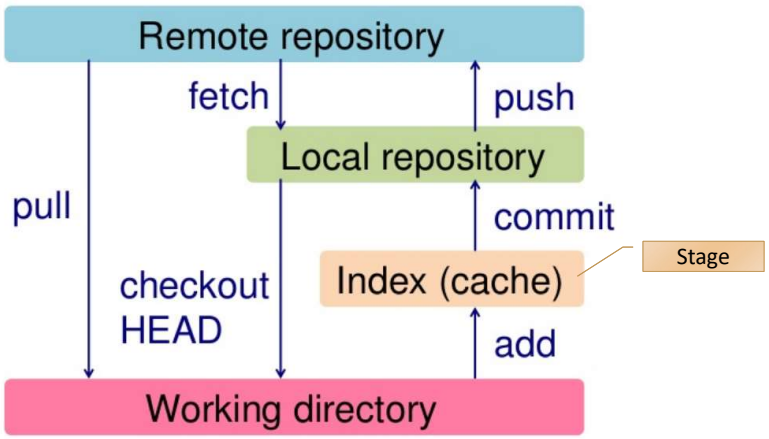


- Git
  - Created by Linus Torvalds in 2005 for development of the Linux kernel, as open-source
  - Distributed
  - DAG storage



©2011-2023 João Cunha, DEIS-ISEC

Git



©2011-2023 João Cunha, DEIS-ISEC

Git: Working on a repository

- Setup a repository
  - **git init:** initialize a new repository
  - **git clone:** copy a repository
  - **git config:** configure repository
- Save changes in files
  - **git add**: adds a change in the working directory to the staging area
  - **git commit**: commits the staged snapshot to the project history
- Inspect the repository
  - **git status:** displays the state of the working directory and the staging area
  - **git log:** command displays committed snapshots
- Undo changes
  - **git checkout:** fetch an old version of a file or the entire working directory from a commit

©2011-2023 João Cunha, DEIS-ISEC

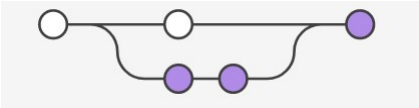
Collaborating - Syncing

- **git remote**: create, view, and delete connections to other repositories
- **git fetch**: imports commits from a remote repository into your local repo
- **git pull**: **fetch** the remote copy of the current branch **+ merge** it into the local copy
- **git push**: transfer commits from your local repository to a remote repo.

©2011-2023 João Cunha, DEIS-ISEC

Branches

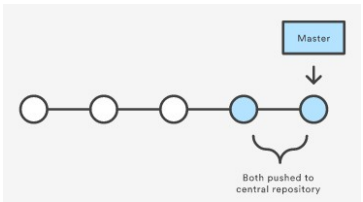
- **git branch:** create, list, rename, and delete branches
- **git checkout:** updates the files in the working directory to match the version stored in that branch
- **git merge:** take the independent lines of development created by git branch and integrate them into a single branch.



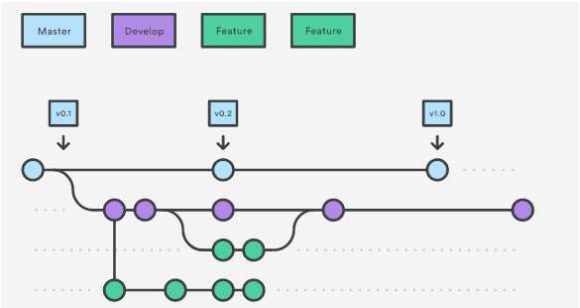
©2011-2023 João Cunha, DEIS-ISEC

Workflows

- Centralized workflow

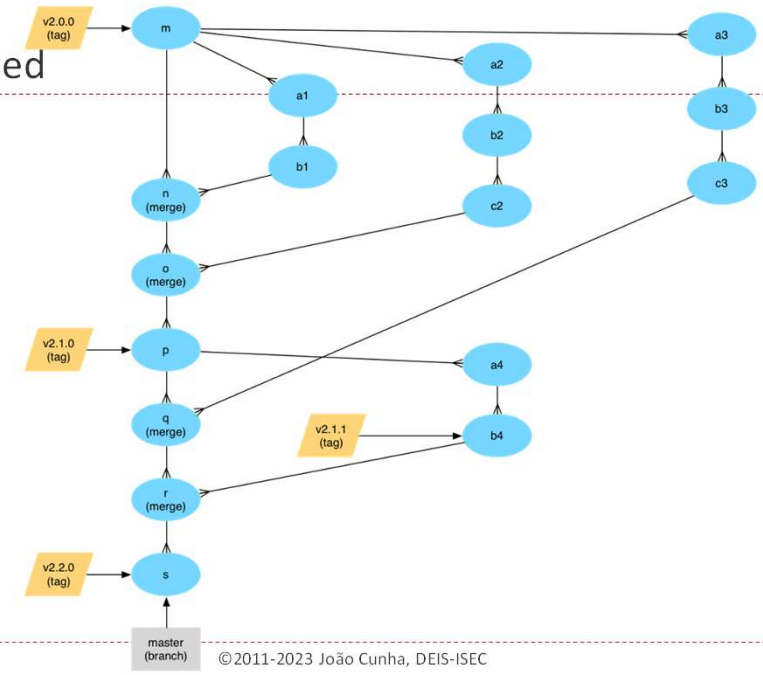


- Feature-based workflow



©2011-2023 João Cunha, DEIS-ISEC

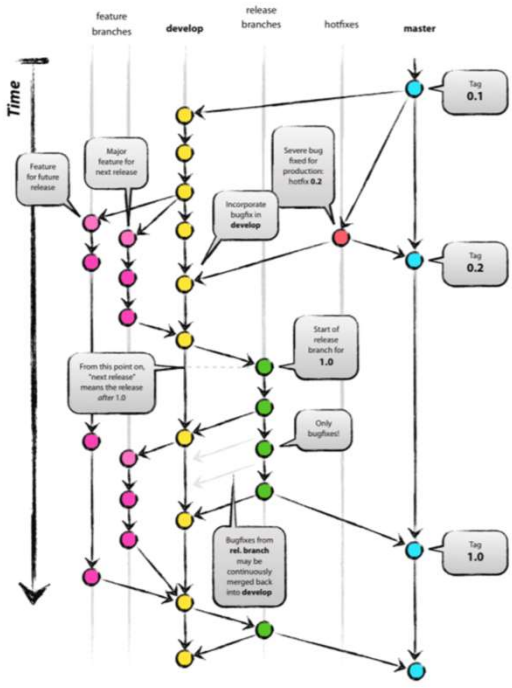
Feature-based



©2011-2023 João Cunha, DEIS-ISEC

51

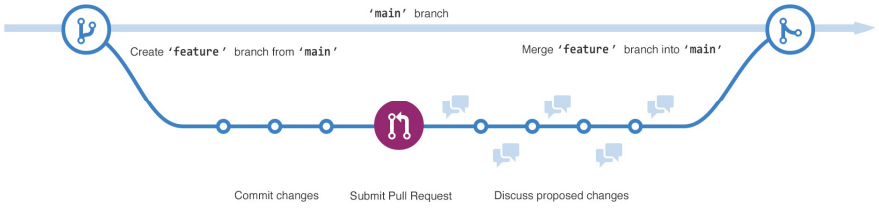
Gitflow workflow



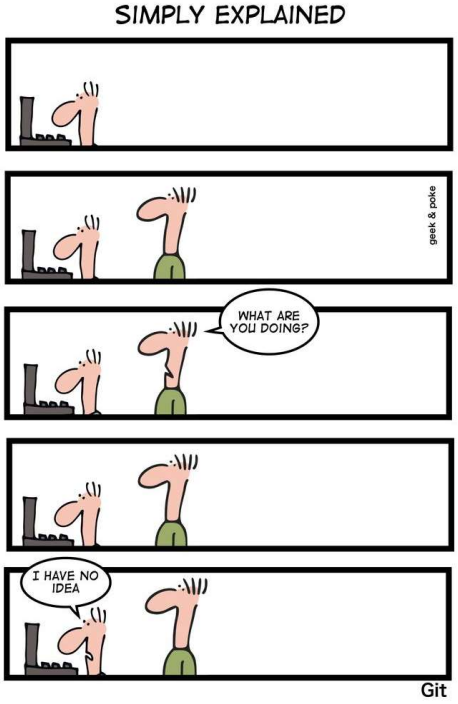
52

Pull Request

1. A developer creates the feature in a dedicated branch in the local repo.
2. The developer pushes the branch to a public repository.
3. The developer files a pull request via the repository.
4. The rest of the team reviews the code, discusses it, and alters it.
5. The project maintainer merges the feature into the official repository and closes the pull request.



©2011-2023 João Cunha, DEIS-ISEC



Further Reading

- GitHub
  - https://guides.github.com/activities/hello-world/
  - https://octoverse.github.com
- Git
  - http://nvie.com/posts/a-successful-git-branching-model/
  - https://www.atlassian.com/git/tutorials
- GitKraken
  - https://www.gitkraken.com/learn-git

    - Some slides adapted from A. Stellman & J.Greene, “Applied Software Project Management” and Ian Sommerville, “Software Engineering”

©2011-2023 João Cunha, DEIS-ISEC

55

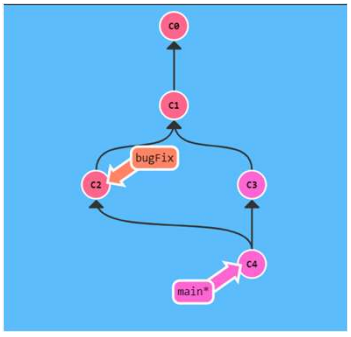
Learn GIT

Oh My GIT!



- https://ohmygit.org/

Learn Git Branching



- https://learngitbranching.js.org/

©2011-2023 João Cunha, DEIS-ISEC

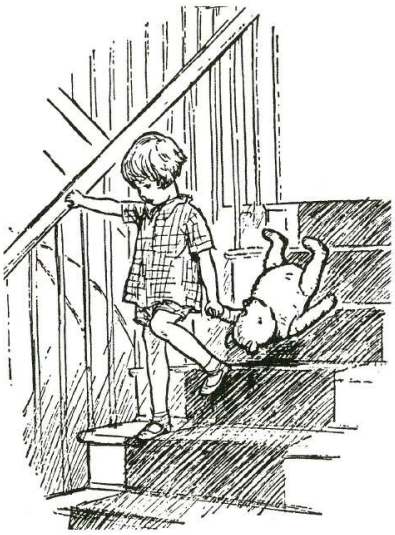
56

3. Traditional Process Models

"Software Project Management"
João Cunha, ISEC (jcunha@isec.pt)

©2011-2023 João Cunha, DEIS-ISEC

57

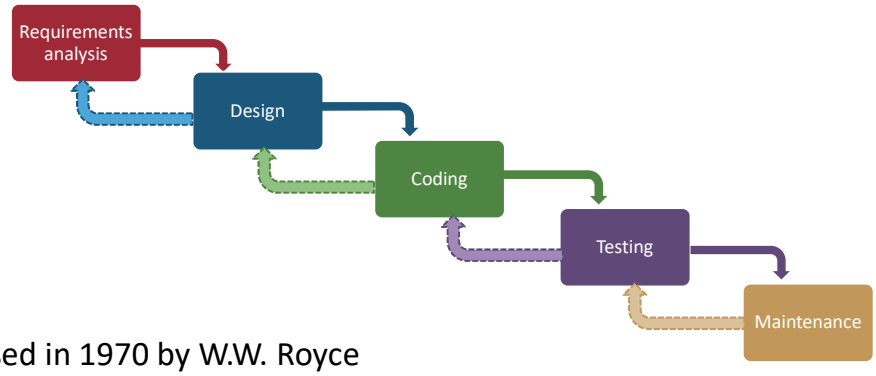


Here is Edward Bear, coming downstairs now, bump, bump, bump on the back of his head, behind Christopher Robin.
It is, as far as he knows, the only way of coming downstairs,
but sometimes he feels that there really is another way,
if only he could stop bumping for a moment and think of it.
And then he feels that perhaps there isn't.

A. A. Milne

58

Waterfall lifecycle



- Proposed in 1970 by W.W. Royce
- Models a systematic sequential approach
  - But real projects rarely follow the sequential flow
  - When something goes wrong, go back to a previous step (expensive)

©2011-2023 João Cunha, DEIS-ISEC

Analysis

- Requirements must be specified in the first step
  - Difficult to accomodate the natural uncertainty at the beginning of the projects
- Models a systematic sequential approach
  - Real projects rarely follow the sequential flow
- The result of each phase is one or more approved documents
  - Document-driven approach – costly iterations
  - Prematurely freezing requirements may mean the system doesn't do what the user wants
- Works well when requirements are well understood, and technologies known

©2008-2020 João Cunha, DEIS-ISEC

4. Agile Software Development
Embrace change

"Software Project Management"

João Cunha, ISEC (jcunha@isec.pt)

©2011-2023 João Cunha, DEIS-ISEC

61

Manifesto for Agile Software Development

We are uncovering better ways of developing
software by doing it and helping others do it.
Through this work we have come to value:

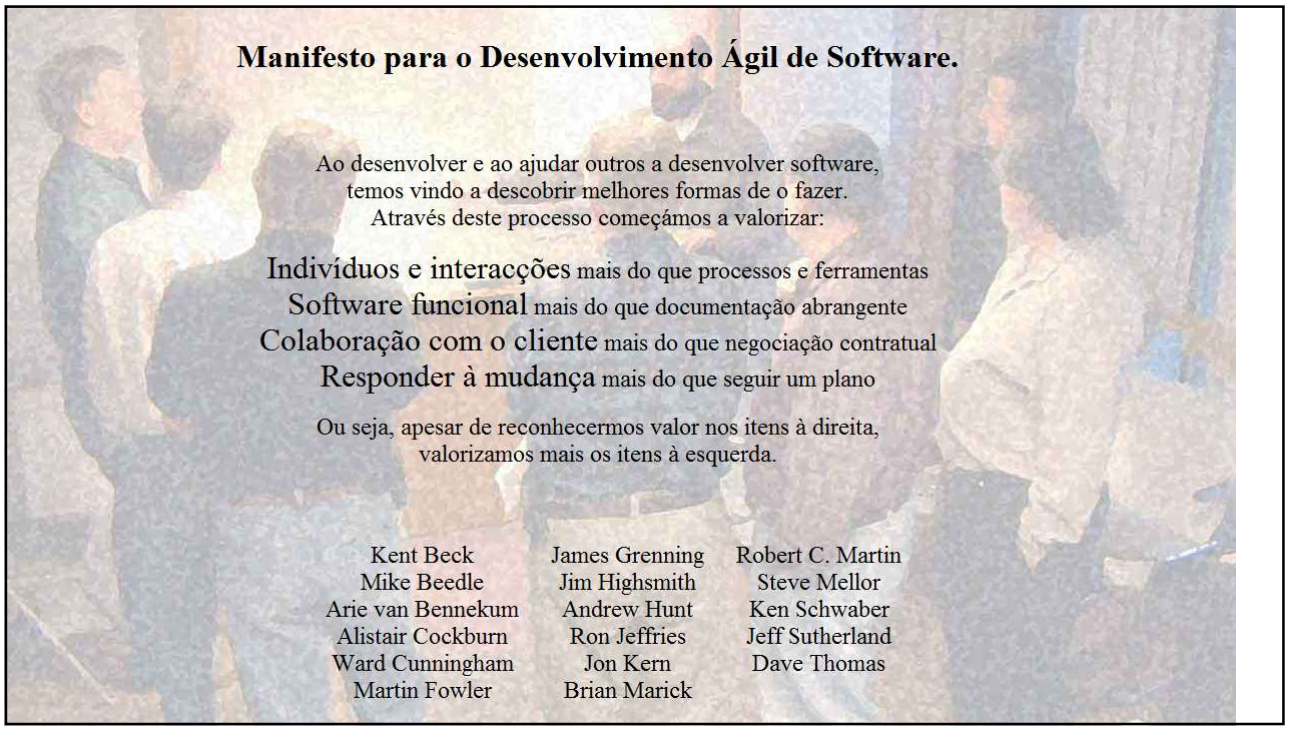
Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on
the right, we value the items on the left more.

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

http://agilemanifesto.org/

62



63

The Agile Principles (1)

1. Our highest priority is to satisfy the customer through **early and continuous delivery** of valuable software.
2. Welcome **changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver **working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter time scale.
4. Business people and developers must **work together** daily throughout the project.
5. Build projects around **motivated individuals**. Give them the environment and support they need and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is **face-to-face conversation**.

©2008-2020 João Cunha, DEIS-ISEC

64

The Agile principles (2)

7. **Working software** is the primary measure of progress.
8. Agile processes promote **sustainable development**. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to **technical excellence** and good design enhances agility.
10. **Simplicity**—the art of maximizing the amount of work not done—is essential.
11. The best architectures, requirements, and designs emerge from **self-organizing teams**.
12. At regular intervals, the team reflects on **how to become more effective**, then tunes and adjusts its behavior accordingly.

©2008-2020 João Cunha, DEIS-ISEC

65

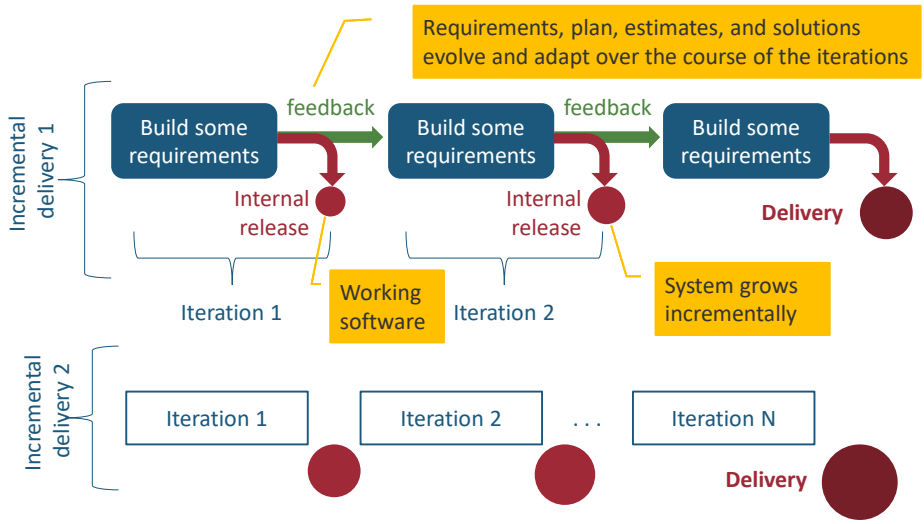
What makes a method Agile?

- Short iterations
- Incremental design
- User involvement
- Minimal documentation
- Informal communication
- Adaptive planning
- Evolutionary delivery

©2008-2020 João Cunha, DEIS-ISEC

66

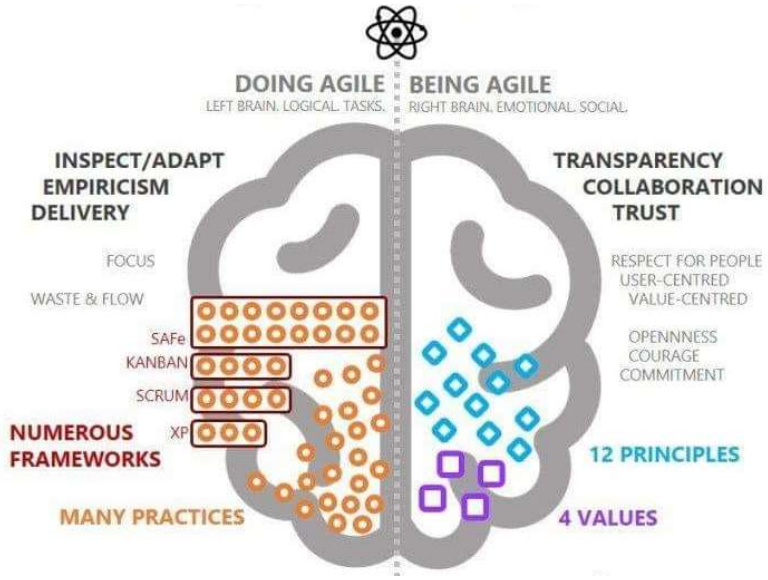
Iterative, evolutionary, adaptive and incremental delivery



©2008-2020 João Cunha, DEIS-ISEC

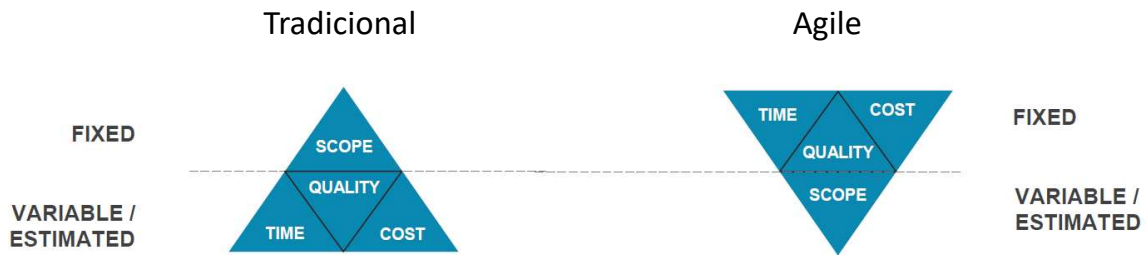
67

Agile Mindset



68

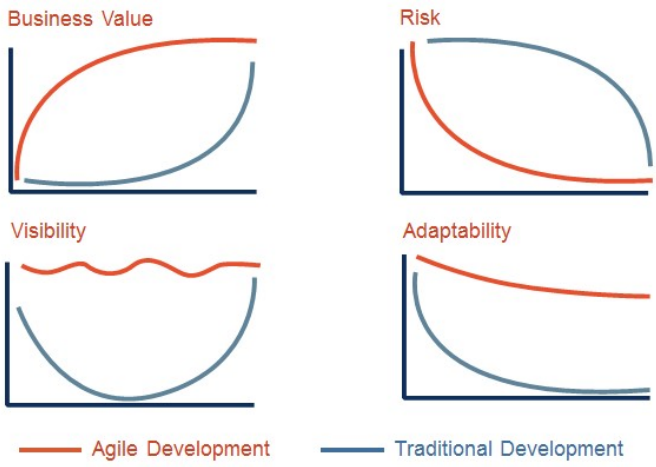
Motivation



©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

69

Motivation



©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

70

Bibliography

- Difference between Agile and Waterfall https://www.youtube.com/watch?v=5RocT_OdQcA
- Agile Alliance https://www.agilealliance.org/

©2011-2023 João Cunha, DEIS-ISEC

5. Scrum

It's about common sense

"Software Development Methodologies"
Ricardo Filipe, ISEC (ricardo.filipe@isec.pt)

©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC



SCRUM

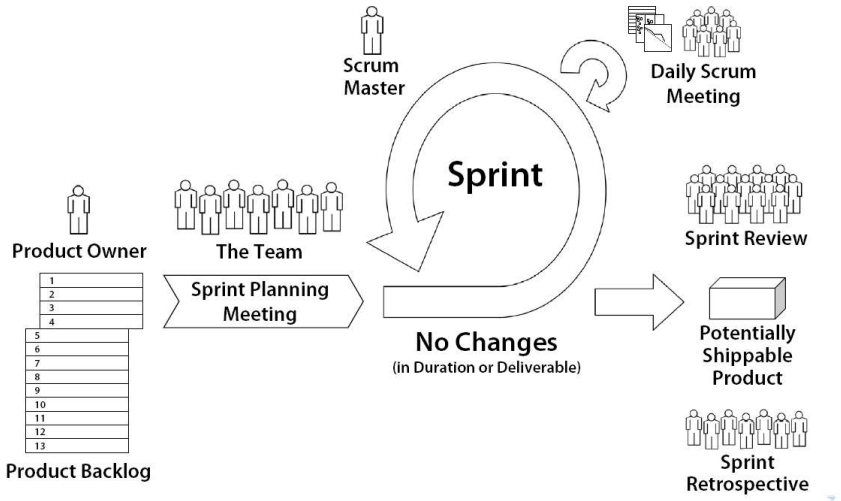
- Scrum is an **iterative, incremental** method for managing the systems development.
- Emphasizes a set of project management **values and practices**.
- **Does not define any specific software development** technique for the implementation phase; it is a wrapper for existing engineering practices.
- Concentrates on **how the team members should function**, promoting self-directed teams and daily team measurement.
- Produces a **potentially shippable** set of functionalities at the end of every iteration.
- Has strong emphasis on **empirical** rather than defined process.

Key practices

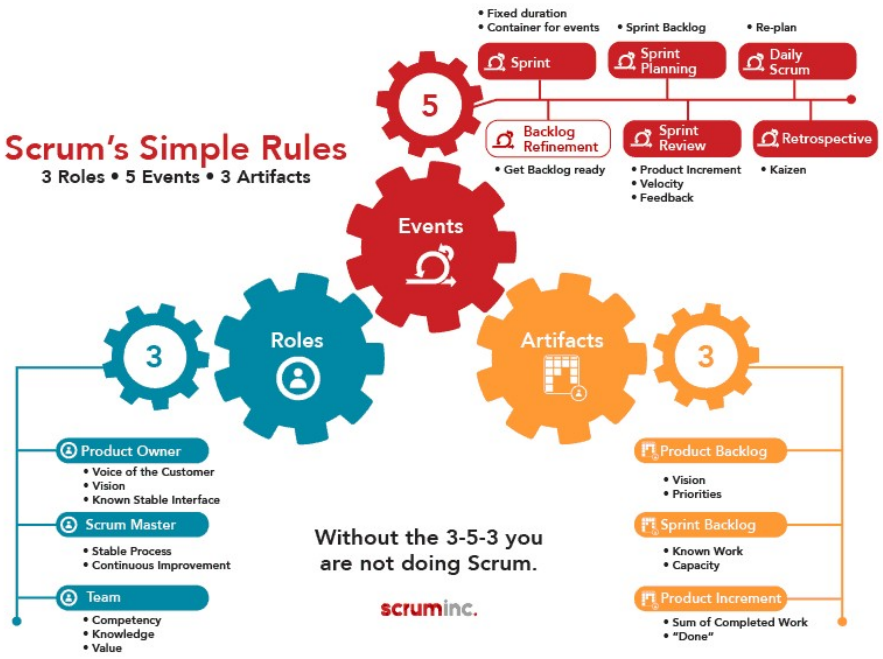
- self-managing team
- no external addition of work to an iteration, once chosen
- daily stand-up meeting
- usually 2 to 4-week iterations
- demo to external stakeholders at end of each iteration
- client-driven adaptive planning

©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Scrum Process flow



©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC



77

77

Scrum framework

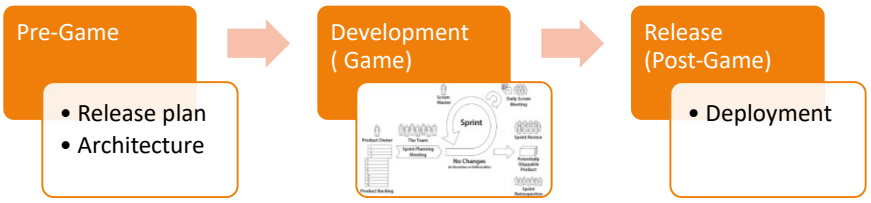
- Roles
  - Product Owner
  - Scrum Master
  - Team
- Artifacts
  - Product Backlog
  - Sprint Backlog
  - Product Increment
- Events
  - Sprint
  - Sprint Planning
  - Daily Scrum Meeting
  - Sprint Review
  - Sprint Retrospective
  - Backlog Refinement

78

©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

78

Scrum Lifecycle



©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Lifecycle

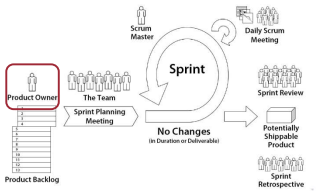
1. Pre-game
   1. Planning
      - Write vision, budget, User Stories, Product Backlog with estimates, exploratory design and prototypes
   2. Staging
      - Define requirements for next release, plan next release, exploratory design and prototypes
2. Development
   - Sprint planning, Sprint, Sprint review and retrospective
3. Release (Post-game)
   - Deployment, documentation, training, …

©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Product Owner

- Owns the vision of what should be produced
- Gets inputs from customers, end-users, team, managers, executives, industry experts, etc.
- Is responsible for creating and prioritizing the Product Backlog
- Monitors the project against the stated goals
- Only one person



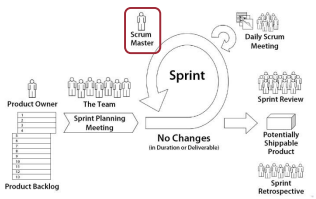


©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Scrum Master

- Responsible for setting the team up for success:
  - Establishes Scrum practices and rules
  - Shields the team from outside disturbances
  - Removes obstacles
- Represents management in the project
- Can be played by a team-member.
- Without a Scrum Master, the team has a high risk of failure.



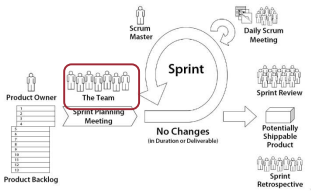


©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

The Development Team

- Until 10 people
- Cross-functional with undifferentiated roles
  - It has all the skills to produce finished product – designers, coders, testers, etc.
  - Everyone contributes based on competency, rather than just job title
- Self-organizing and self-managing
- Responsible for making a commitment and managing itself to hit the goal
- Authority to do whatever is needed to meet the commitment

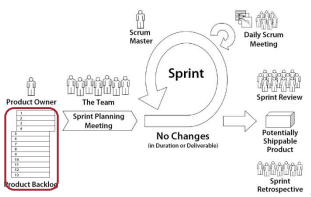




©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Product Backlog

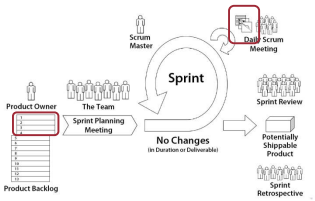
- Prioritized list of work to be performed on a product
  - Requirements oriented or based on user stories
- Anyone can contribute to backlog items
  - Product Owner is responsible
- Product Owner is responsible for prioritization
  - Focus on business value (80% of the value comes from 20% of the features)
  - Reviewed at the start of each sprint or during Backlog Refinement



©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Sprint Backlog

- Lists of the tasks to be executed during the sprint
  - Selected during the Sprint Planning
- Status of the tasks updated daily

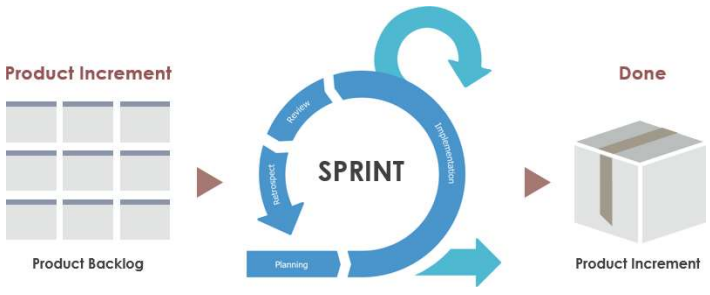


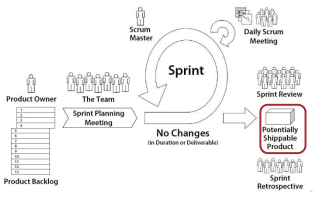
©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

85

Product Increment

- The (working) result at the end of each sprint
- The sum of all the product backlog items completed during a sprint plus the value of the increments of all previous sprints
- The result must work (potentially shippable/releasable) – no presentations.



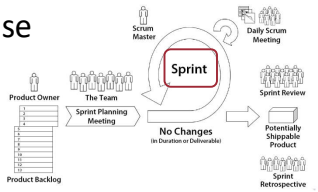


©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

86

Sprint

- Fixed period iteration
  - typically between 2- and 4-weeks
- Includes design, coding, testing, and documentation
- Created “done” work – According to acceptance criteria
- During the Sprint, the Scrum Team keeps committed to the goals and end-date
- The Scrum Team is fully responsible for the work
  - Only the Scrum Team can add or remove items in the Sprint backlog
- Abnormal termination of Sprint may be called by the PO when the Sprint Goal no longer makes sense
- Sprints occur one after another, without any down-time between them.

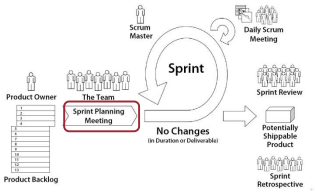


©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

87

Sprint Planning Meeting

- 2-part meeting to define the next Sprint
- Sprint prioritization
  - Product Owner leads, all Stakeholders are present
  - Analyze and evaluate product backlog
  - Select Sprint goal
- Sprint planning
  - Scrum Team and Product Owner (optional)
  - Decide how to achieve sprint goal (design)
  - Create Sprint Backlog (tasks) from Product Backlog (features/user stories)
  - Answer
    - Why is this Sprint valuable?
    - What can be Done this Sprint?
    - How will the chosen work get done?
  - Estimate Sprint Backlog

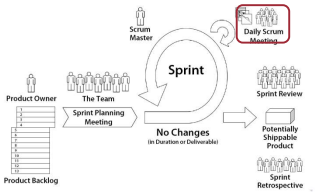


©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

88

Daily Scrum Meeting

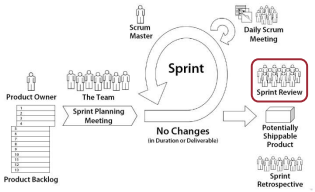
- Short meeting to update each other on progress and problems
  - Daily, stand up, 15 minutes
  - Usually same place, same time, at the beginning of the day
- Everyone answers 3 questions (not necessarily):
  - What have you done since last daily Scrum?
  - What will you do between now and the next daily Scrum?
  - What is getting in the way (blocks) of meeting the iteration goals?
- Scrum Master notes blocks, and afterwards helps resolve them
  - Not for problem solving
  - Other follow-up meetings can be held
- Others (chickens) can attend the meeting but they do not speak.



©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Sprint Review Meeting

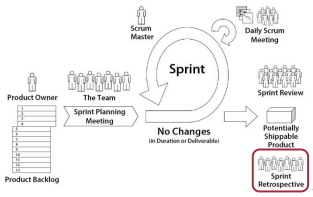
- At the end of the Sprint.
- Product Owner, Team, Scrum Master, and all Stakeholders.
- Demo of what the team has produced.
- The Product Owner gathers feedback from everyone on ways to improve what's been built
  - This feedback is incorporated into the Product Backlog
- Max 4 hours.
- Goal: Inspect product increment.



©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Sprint Retrospective Meeting

- Also at the end of the Sprint
- The Team, Product Owner, and Scrum Master
- Goal: review the way of working and look for ways to improve their effectiveness.
- Mechanism for continuous improvement:
  - What went wrong, what went right, what can be done better
  - Which tools, processes, practices can be used to make it better, more productive and enjoyable
- Might be anonymous



©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

91

Scrum Values



©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

92

Bibliography

- http://www.scrumalliance.org
- https://www.scrum.org/
- https://scrumguides.org/scrum-guide.html

  - Some figures from the slides were taken from these links

©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

6. User Stories

"Software Project Management"
João Cunha, ISEC (jcunha@isec.pt)

©2011-2023 João Cunha, DEIS-ISEC

Agile User Stories

- A user story is an informal, general explanation of a software feature written from the perspective of the end user. Its purpose is to articulate how a software feature will provide value to the customer.
- User story is a show description of a specific customer requirement:
  - **RGB** format: Role; goal; benefit
  - **As a** <type of user> , **I want** <some goal> **so that** <some reason>
- Type of user:
  - Who are we building this for?
  - Who is the persona ?
- Some goal:
  - What is the intention?
  - What are they realy trying to achieve?
- Some reason:
  - What's the overall benefit?
  - What is the big problem that needs to be solved?

©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

INVEST – How a good user story should be

- **I** - Independent (of all others)
- **N** - Negotiable (not a specific contract for features)
- **V** - Valuable (or vertical)
- **E** - Estimable (to a good approximation)
- **S** - Small (so as to fit within an iteration)
- **T** - Testable (in principle, even if there isn't a test for it yet)

©2011-2023 João Cunha, DEIS-ISEC

Three C's - Components of a User Story

- To write a good user story:
  - **Card** – a physycal token, with Title, Role-Goal-Benefit
  - **Conversation** - taking place at a different time, between customers, users, developers, that may include documentation
  - **Confirmation** – the acceptance criteria that should be fulfield and tested to ensure the user requirements have been met and delivered correctly

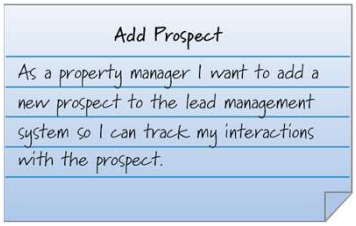
©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

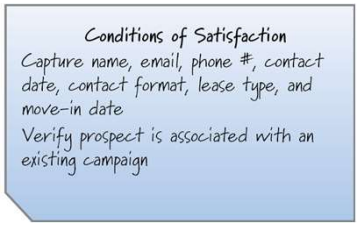
Acceptance Criteria

- Allows to check if the user story is really "done".
- Gathers the requirements and all scenarios that allows to understand that the user story is fully finished.
- Used to:
  - Define scenarios
  - Achieve consensos
  - Serve as a base for testing
  - Manage expectations

©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Agile User Stories and Acceptance Criteria examples





DESCRIPTION:

As a customer, I want to order and pay for the book via a secure web-based form, so that my credit card information is safe.

ACCEPTANCE CRITERIA:

- All mandatory fields must be completed before a customer can submit a form.
- Information from the form is stored in the customer orders database.
- Payment can be made via Amex, Master Card, or Visa credit card.
- The system shall accurately calculate and apply sales tax.
- The system shall accurately calculate and apply shipping charges.
- The customer shall be able to verify the accuracy of the order.
- An acknowledgment email is sent to the customer submitting the form.
- Protection against spam is working.
- The code should be deployed and running in Staging environment.

©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Given/When/Then Acceptance Criteria

- **Scenario:** (name of the behavior)
  - **Given** a context
  - **When** an event happens
  - **Then** there is an outcome
  - **And** (may be used to continue any of the 3 previous statements)

©2011-2023 João Cunha, DEIS-ISEC

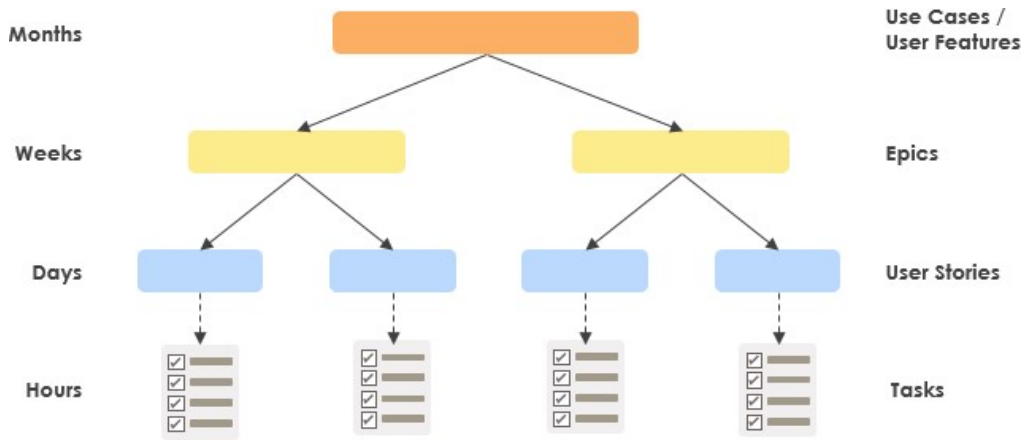
GWT examples

User story: As a user, I want to be able to request the cash from my account at an ATM so that I will be able to receive the money from my account quickly and in different places.	
Scenario 2	Requesting the cash from an overdrawn account
Given	The account is overdrawn
And	The card is valid
When	The customer requests the cash
Then	Ensure the rejection message is displayed
And	Ensure cash isn't dispensed

User story: As a user, I want to be able to recover the password to my account, so that I will be able to access my account in case I forgot the password.	
Scenario	Forgot password
Given	The user navigates to the login page
When	The user selects <*forgot password*> option
And	Enters a valid email to receive a link for password recovery
Then	The system sends the link to the entered email
Given	The user receives the link via the email
When	The user navigates through the link received in the email
Then	The system enables the user to set a new password

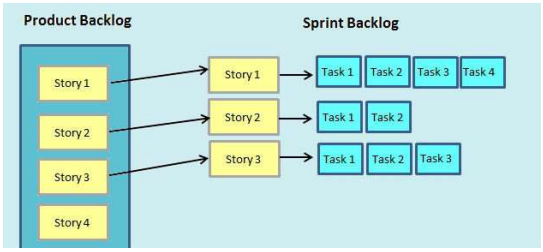
©2011-2023 João Cunha, DEIS-ISEC

Use Cases, Epics, User Stories, Tasks

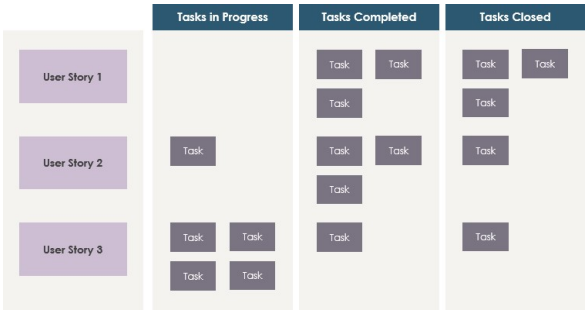


©2011-2023 João Cunha, DEIS-ISEC

Product and Sprint Backlog



Sprint Backlog – Scrum Board



©2011-2023 João Cunha, DEIS-ISEC

Further Reading

- How to write a Good User Story https://www.youtube.com/watch?v=P3hbbKnzm4A
- https://www.atlassian.com/agile/project-management/user-stories
- https://www.agilealliance.org/glossary/user-stories/
- Alexsoft, Acceptance Criteria for User Stories: Purposes, Formats, Examples, and Best Practices, 18/5/2021 (https://www.altexsoft.com/blog/business/acceptance-criteria-purposes-formats-and-best-practices/)

©2011-2023 João Cunha, DEIS-ISEC

7. Software Testing

"Software Project Management"
João Cunha, ISEC (jcunha@isec.pt)

©2011-2023 João Cunha, DEIS-ISEC

Program testing

- The purpose of testing is to show that a program does what it is intended to do and to discover program defects before it is put into use.
- When you test software, you execute a program using artificial data.
- You check the results of the test run for errors, anomalies or information about the program's non-functional attributes.
- Tests can reveal the presence of errors NOT their absence.
- Testing is part of a more general verification and validation (V&V) process.

©2011-2023 João Cunha, DEIS-ISEC

Reviews and testing

- **Software reviews** Concerned with analysis of the static system representation to discover problems (static verification)
  - May be supplemented by tool-based document and code analysis.
- **Software testing** Concerned with exercising and observing product behaviour (dynamic verification)
  - The system is executed with test data and its operational behaviour is observed.

©2011-2023 João Cunha, DEIS-ISEC

Reviews and testing

- Reviews and testing are complementary and not opposing verification techniques.
- Both should be used during the V&V process.
- Reviews can check conformance with a specification but not conformance with the customer's real requirements.
- Reviews cannot check non-functional characteristics such as performance, usability, etc.

©2011-2023 João Cunha, DEIS-ISEC

1. Unit testing

- Unit testing is the process of testing individual components in isolation.
- It is a defect testing process.
- Units may be:
  - Individual functions or methods within an object
  - Object classes with several attributes and methods
  - Composite components with defined interfaces used to access their functionality.

©2011-2023 João Cunha, DEIS-ISEC

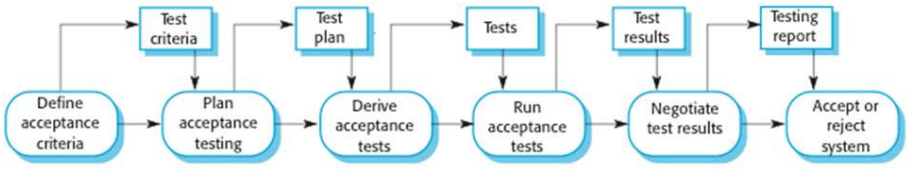
2. Integration testing

- In integration testing individual software modules are combined and tested as a group.
- It occurs after unit testing and before system or acceptance testing.
- In **incremental integration** testing, the developers integrate the modules one by one
- **Big bang integration** is another integration testing technique, where all the modules are integrated in one shot (of course, you should avoid this!)

©2011-2023 João Cunha, DEIS-ISEC

3. Acceptance testing

- Acceptance testing is a user testing process where the aim is to decide if the software is good enough to be deployed and used in its operational environment.



©2011-2023 João Cunha, DEIS-ISEC

Agile – Acceptance Criteria

- Conditions that must be satisfied for a product, user story, or increment of work to be accepted.
- Expressed as a set of statements:
  - Clear, so that everyone understands them
  - Concise, so that there's no ambiguity
  - Testable or verifiable
  - Focused on providing customer-delighting results
- Acceptance criteria complement User stories

©2011-2023 João Cunha, DEIS-ISEC

Examples of acceptance criteria

- For a checkout page on a store's website:
  - PayPal, Google Pay, Apple Pay, and all major credit cards can be used to complete the transaction
  - Shopping cart item(s) are displayed
  - Items can be deleted from the shopping cart
  - User is prompted to log in if they aren't already
- For an emailed water bill statement:
  - Displays amount due
  - Displays due date
  - Displays possible late fee
  - Allows user to log in and pay easily
  - Lets user know how they can contact the office with questions or concerns

©2011-2023 João Cunha, DEIS-ISEC

4. Usability testing

- Usability testing refers to evaluating a product or service by testing it with representative users.
- Typically, during a test, participants will try to complete typical tasks while observers watch, listen and take notes.
- The goal is to identify any usability problems, collect qualitative and quantitative data and determine the participant's satisfaction with the product.

©2011-2023 João Cunha, DEIS-ISEC

Usability test plan

- Should include:
  - **Scope** - Indicate what you are testing. E.g. the navigation and content.
  - **Purpose** - Identify the concerns, questions, and goals for this test. Example: "Can users navigate to important information from the prototype's home page?"
  - **Participants** - Number and types of participants
  - **Scenarios** - describe the stories and context behind why a specific user or user group comes to your application. They note the goals and questions to be achieved and sometimes define the possibilities of how the user(s) can achieve them on the application. Example: You are traveling to Bragança for your job next week and you want to check on the amount you can be reimbursed for meals and other expenses
  - **Metrics** (e.g., successful completion rates, error rates, time on task)

©2011-2023 João Cunha, DEIS-ISEC

Further reading

- Andrew Stellman and Jennifer Greene, *Applied Software Project Management*, O'Reilly, 2006. [Section 8.1]
- *Usability testing*, online: https://www.usability.gov/how-to-and-tools/methods/usability-testing.html
- Some slides adapted from A. Stellman & J.Greene, "Applied Software Project Management" and Ian Sommerville, "Software Engineering"
- Everything You Need to Know About Acceptance Criteria, BY SCRUM ALLIANCE, https://resources.scrumalliance.org/Article/need-know-acceptance-criteria

©2011-2023 João Cunha, DEIS-ISEC

8. Unit Test - JUnit

"Software Project Management"
João Cunha, ISEC (jcunha@isec.pt)

©2011-2023 João Cunha, DEIS-ISEC

Why create a test suite?

- How to test your code
  1. Do ad hoc testing (running whatever tests occur at the moment)
  2. Build a test suite (a thorough set of tests that can be run at any time)
- Disadvantages of a test suite
  - It's a lot of extra programming
    - However, a good test framework can help quite a lot
  - No time to do all that extra work
    - Experiments repeatedly show that test suites reduce debugging time more than the amount spent building the test suite
- Advantages of a test suite
  - Reduces total number of bugs in delivered code
  - Makes code much more maintainable and refactorable

©2011-2023 João Cunha, DEIS-ISEC

JUnit

- JUnit is a unit testing framework for the Java programming language.
- It is an instance of the xUnit architecture for unit testing frameworks
- JDK
  - Junit 4 requires Java 5 or higher.
  - Junit 5 requires Java 8 or higher.

©2011-2023 João Cunha, DEIS-ISEC

Terminology

- A **test fixture** sets up the data (both objects and primitives) that are needed to run tests
  - Example: If you are testing code that updates an employee record, you need an employee record to test it on
- A **unit test** is a test of a *single* class
- A **test case** tests the response of a single method to a particular set of inputs
- A **test suite** is a collection of test cases
- A **test runner** is software that runs tests and reports results

- An **integration test** is a test of how well classes work together
  - JUnit provides some limited support for integration tests

©2011-2023 João Cunha, DEIS-ISEC

@Test

- A test method is annotated with @Test, takes no parameters, and returns no result
- All the usual assert*XXX* methods can be used

```
@Test
public void sum() {
    assertEquals(15, program.sum(array));
    assertTrue(program.min(array) > 0);
}
```


©2011-2023 João Cunha, DEIS-ISEC

Assert methods

- Each assert method has parameters like these:
  - message, expected-value, actual-value (JUnit 4)
  - expected-value, actual-value, message (JUnit 5)
- Assert methods dealing with floating point numbers get an additional argument, a tolerance
- Each assert method has an equivalent version that does not take a message

©2011-2023 João Cunha, DEIS-ISEC

Assert methods

- assertTrue(Boolean test)
  - Asserts that the supplied condition is true
- assertFalse(Boolean test)
- assertNull(Object object)
  - Asserts that object is null.
- assertNotNull(Object object)
- assertEquals(Object expected, Object actual)
  - Asserts that expected and actual are equal. Uses equals() method
- assertSame(Object expected, Object actual)
  - Asserts that expected and actual do refer to the same object. Uses == operator
- assertNotSame(Object expected, Object actual)

©2011-2023 João Cunha, DEIS-ISEC

Exceptions

- JUnit 5
  - assertThrows(Class<T> expectedType, Executable executable)

```
@Test
public void whenDerivedExceptionThrown_thenAssertionSucceds() {
    Exception exception = assertThrows(RuntimeException.class, () -> {
        Integer.parseInt("1a");
    });

    String expectedMessage = "For input string";
    String actualMessage = exception.getMessage();

    assertTrue(actualMessage.contains(expectedMessage));
}
```

- JUnit 4
  - @Test(expected = Class<T> expectedType)

```
@Test(expected = NullPointerException.class)
public void whenExceptionThrown_thenExpectationSatisfied() {
    String test = null;
    test.length();
}
```


©2011-2023 João Cunha, DEIS-ISEC

@BeforeClass and @AfterClass (JUnit4)
@BeforeAll and @AfterAll (JUnit5)

- If you wish, you can declare *one* method to be executed *just once,* when the class is first loaded
- This is for expensive setup, such as connecting to a database

```
@BeforeClass
public static void setUpClass() throws Exception {
        // one-time initialization code
}
```

- If you wish, you can declare *one* method to be executed *just once,* to do cleanup after all the tests have been completed

```
@AfterClass
public static void tearDownClass() throws Exception {
        // one-time cleanup code
}
```


©2011-2023 João Cunha, DEIS-ISEC

@Before and @After (JUnit4)
@BeforeEach and @AfterEach (JUnit5)

- You can define one or more methods to be executed before each test; typically, such methods initialize values, so that each test starts with a fresh set

```
@Before
public void setUp() {
    program = new MyProgram();
    array = new int[] { 1, 2, 3, 4, 5 };
}
```

- You can define one or more methods to be executed after each test; typically, such methods release resources, such as files

```
@After
public void tearDown() {
}
```


©2011-2023 João Cunha, DEIS-ISEC

Example (JUnit4)

```
import org.junit.*;
import static org.junit.Assert.*;
public class MoneyTest {
   private Money f12CHF;
   private Money f14CHF;

   @Before
   public void setUp() {        // create the test data
    f12CHF = new Money(12, "CHF");     // - the fixture
    f14CHF = new Money(14, "CHF");
   }
   @Test
   public void testadd() {         // create the test data
    Money expected = new Money(26, “CHF”);
    assertEquals(“amount not equal”, expected, f12CHF.add(f14CHF));
   }
   ...
}
```

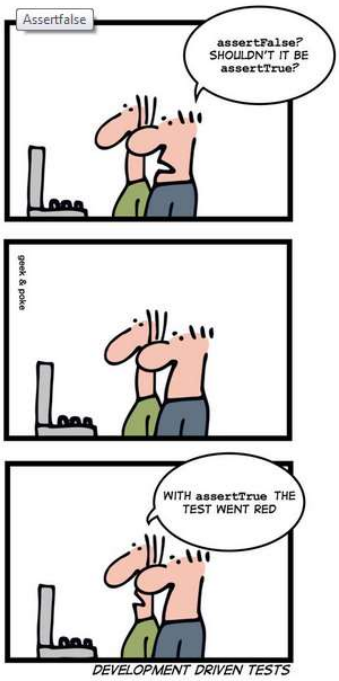

©2011-2023 João Cunha, DEIS-ISEC

JUnit5 example: Test Class

```
public class Calculator {
    public int multiply(int a, int b) {
        return a * b;
    }
}
```

```
class CalculatorTest {
    Calculator calculator;
    @BeforeEach
    void setUp() {
        calculator = new Calculator();
    }
    @Test
    void testMultiply() {
        assertEquals(20, calculator.multiply(4, 5),
                "Regular multiplication should work");
    }
}
```


©2011-2023 João Cunha, DEIS-ISEC



129

129

More Information

- JUnit
  - https://junit.org/junit5/
  - Tutorials
    - https://www.guru99.com/junit-tutorial.html
    - https://www.vogella.com/tutorials/JUnit/article.html
- PHPUnit
  - http://www.phpunit.de/

Some slides adapted from A. Stellman & J.Greene, “Applied Software Project Management” and Ian Sommerville, “Software Engineering”

130

©2011-2023 João Cunha, DEIS-ISEC

130

9. Risk Management

"Software Project Management"
João Cunha, ISEC (jcunha@isec.pt)

©2011-2023 João Cunha, DEIS-ISEC

Risk Plan

- A risk plan is a list of all risks that threaten the project, along with a plan to mitigate some or all of those risks.
  - The project manager selects team members to participate in a risk planning session:
    - The team members brainstorm potential risks
    - The probability and impact of each risk is estimated
    - A risk plan is constructed

©2011-2023 João Cunha, DEIS-ISEC

- Project Risk is an uncertain event or condition that, if it occurs, has a positive or negative effect on, at least, one project objective, such as time, cost, scope or quality [PMBOK Guide 2004].
- RISK is the possibility of suffering loss

©2011-2023 João Cunha, DEIS-ISEC

133

Risk management

- Risk management is concerned with identifying risks and drawing up plans to minimise their effect on a project.
- A risk is a probability that some adverse circumstance will occur
  - **Project risks** affect schedule or resources;
  - **Product risks** affect the quality or performance of the software being developed;
  - **Business risks** affect the organisation developing or procuring the software.

©2011-2023 João Cunha, DEIS-ISEC

134

The risk management process

1. **Risk identification**
   - Identify project, product and business risks;
2. **Risk analysis**
   - Assess the likelihood and consequences of these risks;
3. **Risk planning**
   - Draw up plans to avoid or minimise the effects of the risk;
4. **Risk monitoring**
   - Monitor the risks throughout the project;

©2011-2023 João Cunha, DEIS-ISEC

1. Risk identification

- Team activity
- A checklist of common risks may be used to identify risks in a project
  - Technology risks.
  - People risks.
  - Organisational risks.
  - Requirements risks.
  - Estimation risks.

©2011-2023 João Cunha, DEIS-ISEC

Threshold of Success (ToS)

- The minimum set of conditions that must be met for the project to be considered successful
- If the team fails to meet **even one** of the conditions, then the project is a **failure**
- Just define 2-3 **SMART** goals.
  - **S**hort/**S**pecific, **M**easurable, **A**chievable, **R**elevant, and **T**ime bound.

©2011-2023 João Cunha, DEIS-ISEC

How to create a ToS

- First create a picture of failure, then convert failure into success.

Ex:

- The project fails if:
  - Essential features are not ready by the first release date.
  - Team members are dissatisfied or bored with their jobs.
- The project is successful if:
  - By the first release date, all "Must Have" features are implemented and pass acceptance tests with no known critical defects.
  - All team members give average score of 5 or better on a job satisfaction survey taken quarterly.

©2011-2023 João Cunha, DEIS-ISEC

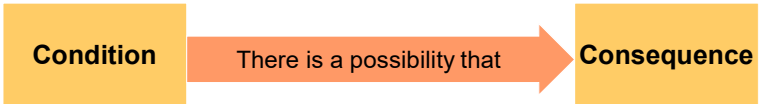
Risk Statement

- Risks are captured in statements to:
  - Help understand those issues/concerns that might stop the project from being successful, i.e, fail to meet the ToS.
  - Provide consistency in how the information is displayed within and across projects.
  - Understand and follow project concerns.
  - Better communicate concerns to project stakeholders
- Statements create a concise description of particular issues or concerns that can be understood and acted upon.

©2011-2023 João Cunha, DEIS-ISEC

Guidelines

- Consider and capture those **fact-based conditions (true now!)** that are causing concern for a potential loss to the project;
- followed by a brief description of the **potential negative consequences** of these conditions.



©2011-2023 João Cunha, DEIS-ISEC

What does a good statement look like?

- **Condition**: a single phrase or sentence that briefly describes the key circumstances, situations, etc., that have caused concern, doubt, anxiety or uncertainty.
- **Consequence**: a single phrase or sentence that describes the key negative outcome(s) that might result.
- Example:
  - **Tight schedules are causing people to work too much overtime**; (there is a possibility that this) **might cause project burnout and lower team morale**.

©2011-2023 João Cunha, DEIS-ISEC

141

Examples of different risk types

Risk type	Possible risks: condition;	consequence
Technology	The database used in the system cannot process as many transactions per second as expected;	performance requirements might not be met
People	Only one team member is expert in swing, and there are many screens to develop;	might delay GUI development, delaying the whole project.
Organizational	Organization has financial problems;	might force reductions in the project budget.
Tools	The code generated by software code generation tools is inefficient;	might compromise software quality.
Requirements	Customers fail to understand the impact of requirements changes;	requested changes might require major design rework .
Estimation	The team has no experience in software estimation;	might case underestimation, delaying the project in an uncontrolled way.

©2011-2023 João Cunha, DEIS-ISEC

142

2. Risk analysis

- Assess probability and impact of each risk.
- Risk ***impact*** might be catastrophic(5), serious(3), or tolerable(1).
- ***Probability*** may be very low(1), low, moderate, high or very high(5).
- Prioritize risks according to ***probability x impact***

©2011-2023 João Cunha, DEIS-ISEC

Risk matrix

P x I		Impact				
		Tolerable (1)		Serious (3)		Catastrophic (5)
Probability	Very low (1)	Low (1)	Low (2)	Low (3)	Low (4)	Low (5)
	Low (2)	Low (2)	Low (4)	Low (6)	Low (8)	Medium (10)
	Moderate (3)	Low (3)	Low (6)	Low (9)	Medium (12)	Medium (15)
	High (4)	Low (4)	Low (4)	Medium (12)	Medium (16)	High (20)
	Very high (5)	Low (5)	Medium (10)	Medium (15)	High (20)	High (25)

©2011-2023 João Cunha, DEIS-ISEC

3. Risk planning

- Consider the top risks (e.g. with probability x impact >= 20) and develop a strategy to mitigate those risks.

- **Avoidance strategies** (*Plano de Prevenção*)
  - The probability that the risk will arise is reduced;
- **Minimisation strategies** *(Plano de Minimização)*
  - The impact of the risk on the project or product will be reduced;
- **Contingency plans** *(Plano de Contingência)*
  - If the risk arises, contingency plans are to deal with that problem;

©2011-2023 João Cunha, DEIS-ISEC

4. Risk monitoring

- Assess each identified risks **regularly** to decide whether or not it is becoming less or more probable.
- Also assess whether the effects of the risk have changed.
- Each key risk should be discussed at management progress meetings.

©2011-2023 João Cunha, DEIS-ISEC

Further reading

- Ian Sommerville, *Software Engineering 9*, Pearson Education, 2011. [Section 22.1]
- Some slides adapted from Ian Sommerville, “Software Engineering”

©2011-2023 João Cunha, DEIS-ISEC

147

10. Estimation

“Software Project Management”

João Cunha, ISEC (jcunha@isec.pt)

©2011-2023 João Cunha, DEIS-ISEC

148

What is estimation?

- The project manager must set expectations about the time required to complete the software among the stakeholders, the team, and the organization's management.
- If those expectations are not realistic from the beginning of the project, the stakeholders will not trust the team or the project manager.

©2011-2023 João Cunha, DEIS-ISEC

What is an estimate?

- "A tentative evaluation or rough calculation."
- "A preliminary calculation of the cost of a project. "
- "A judgment based upon one's impressions; opinion. "
  - Source: The American Heritage Dictionary, Second College Edition, 1985
- It's not:
  - A commitment or a plan to meet a target

©2011-2023 João Cunha, DEIS-ISEC

Why estimate?

- To support
  - Contract negotiation – cost and schedule
  - Project planning
  - Project control
- "... determine whether a project's targets are realistic enough to allow the project to be controlled to meet them."
  - Source: Software Estimation – Demystifying the Black Art, 2006

©2011-2023 João Cunha, DEIS-ISEC

Elements of a Sound Estimate

- To generate a sound estimate, a project manager must have:
  - A work breakdown structure (WBS), a product breakdown structure (PBS), or a list of tasks which, if completed, will produce the final product
  - An effort estimate for each task
  - A list of assumptions which were necessary for making the estimate
  - Consensus among the project team that the estimate is accurate

©2011-2023 João Cunha, DEIS-ISEC

How good an estimator are you?

- For each question, fill in the upper and lower bounds that, in your opinion, give you a 90% chance of including the correct value
- What is the surface temperature of the Sun?
  - Low estimate:
  - High estimate:
- How long does it take to go from ISEC in Coimbra to ISEL in Lisbon?
  - Low estimate:
  - High estimate:

©2011-2023 João Cunha, DEIS-ISEC

153

Influencing factors

- Which factors did you took in consideration?
  - Confidence level
  - Knowledge base/ historical data
  - Personal experience
- For the travelling time estimation
  - Transport (train, car, bus, ...)
  - Route (highways, secondary roads, ...)
  - Traffic hours
  - Complying with driving rules (speed limit, ...)
  - Resting/eating pauses
  - Unexpected conditions (weather, accidents, ...)
- Did you make any assumptions?
- Did you consider several options?

©2011-2023 João Cunha, DEIS-ISEC

154

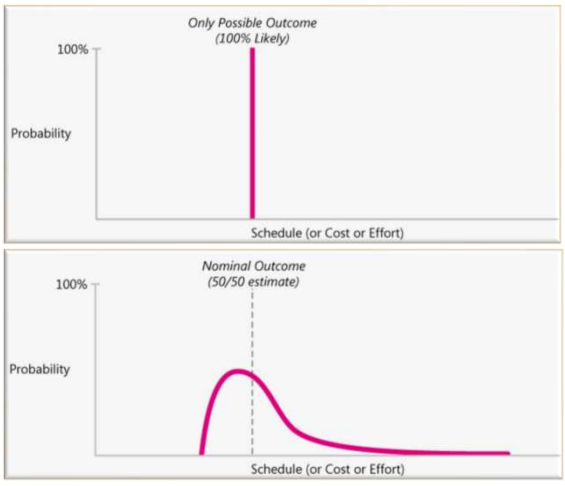
Assumptions Make Estimates More Accurate

- Team members make assumptions about the work to be done in order to deal with incomplete information
  - Any estimate must be based on a decision that has not yet been made; team members can assume the answer for the sake of the estimate
  - Assumptions must be written down so that if they prove to be incorrect and cause the estimate to be inaccurate, everyone understands what happened
  - Assumptions bring the team together very early on the project so they can make progress on important decisions that will affect development

©2011-2023 João Cunha, DEIS-ISEC

155

Estimation as probability statements



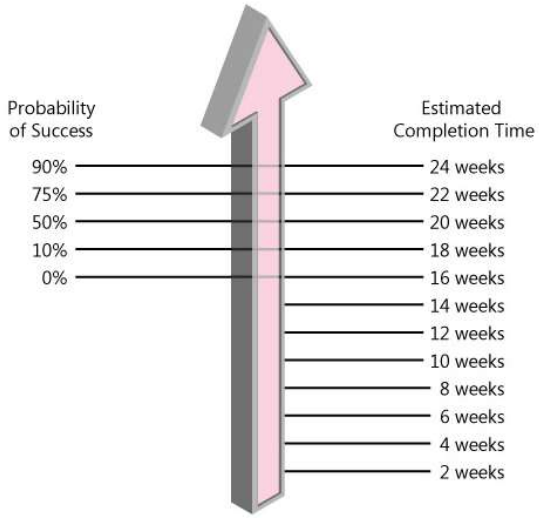
Single-point estimates assume 100% probability of the actual outcome equaling the planned outcome **UNREALISTIC!**

Project outcomes: There is a limit to how well a project can go but no limit to how many problems can occur **REALISTIC!**

©2011-2023 João Cunha, DEIS-ISEC

156

Estimation as probability statements



157

Main issues when estimating

- Complexity of systems
  - Non-repeatable work
- Bias torward underestimation
  - When overestimation occurs there's Parkinson's Law
- Estimates are in fact goals
  - When estimating we manipulate the output to achive the goal
- Inaccurate information about organization capabilities
  - Missing or unused historical data
- Unclear requirements
  - Stakeholders with different understanding of what to develop
  - Hidden assumptions

158

WBS – work breakdown structute

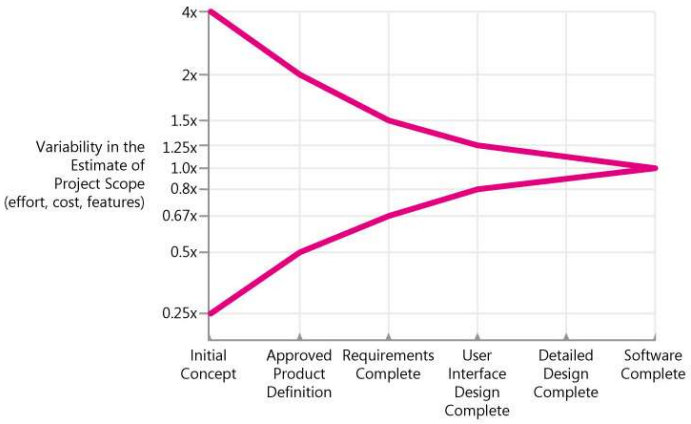
- Need to decompose the project into manageable chunks
- Hierarchical list of project work activities

- Ex:
  - User Story 1
    - Task11
    - Task12
  - User Story 2
    - Task21
    - Task22

©2011-2023 João Cunha, DEIS-ISEC

When to estimate?

- The Cone of Uncertainty



Source: *Software Estimation – Demystifying the Black Art*, 2006

©2011-2023 João Cunha, DEIS-ISEC

How to estimate?

- Estimation techniques
  - Wild Altogether Guess
  - Experience - experts judgment
  - Wideband Delphi
    - experts judgement in groups
    - Variants: Estimation game, Planning poker
  - Estimation by Analogy
  - Function Points Analysis
    - Variants: Use Case Points, Object Points, COSMIC, ...
  - COCOMO II
  - Bottom-up
    - estimate each component
  - ...

©2011-2023 João Cunha, DEIS-ISEC

What and how to estimate in agile?

- Product Backlog -> User Stories -> Story Points
- Sprint Backlog -> Tasks -> Hours/Days

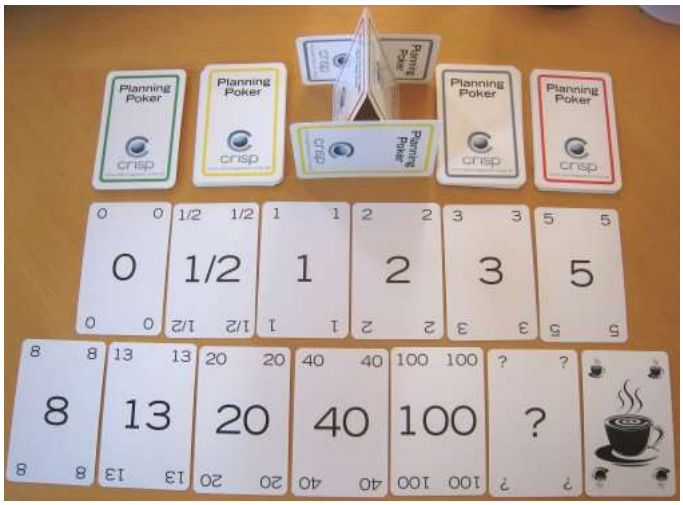
©2011-2023 João Cunha, DEIS-ISEC

Story points

- Story points are a unit of measure to estimate the effort required to complete a specific task, user story, or feature.
- They are a **relative measure** of complexity and effort rather than a precise measure of time.
- Help development teams plan and prioritize their work more effectively by providing a common understanding of the work involved.
- Compare items based on:
  - **risk** or **uncertainty** associated with the tasks
  - the **amount of work** from team's experience with similar tasks
  - the tasks **complexity**
- Estimate the story points using e.g. Planning Poker, based on the Fibonacci sequence (0, 0.5, 1, 2, 3, 5, 8, 13, 20, 40, 100), or T-Shirt Size

©2011-2023 João Cunha, DEIS-ISEC

How to estimate – Planning Poker



©2011-2023 João Cunha, DEIS-ISEC

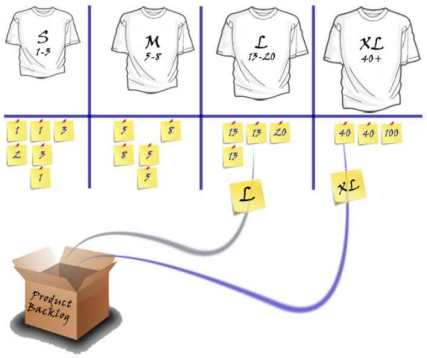
How to estimate – Planning Poker

1. Form a group of no more than 10 estimators and a moderator. The product owner or client manager cannot be an estimator.
2. Each estimator gets a deck of cards: 0, 0.5, 1, 2, 3, 5, 8, 13, 20, 40, 100 and ?.
3. The moderator reads the description of the user story or task. The product owner answers questions from the estimators.
4. Every estimator selects an estimate card and places it face down on the table. After all estimates are in, the cards are flipped over.
5. If the estimates vary widely, the owners of the high and low estimates discuss the reasons for diverging. All estimators should participate in the discussion, 3 minutes max.
6. Repeat from step 4 until the estimates converge.

©2011-2023 João Cunha, DEIS-ISEC

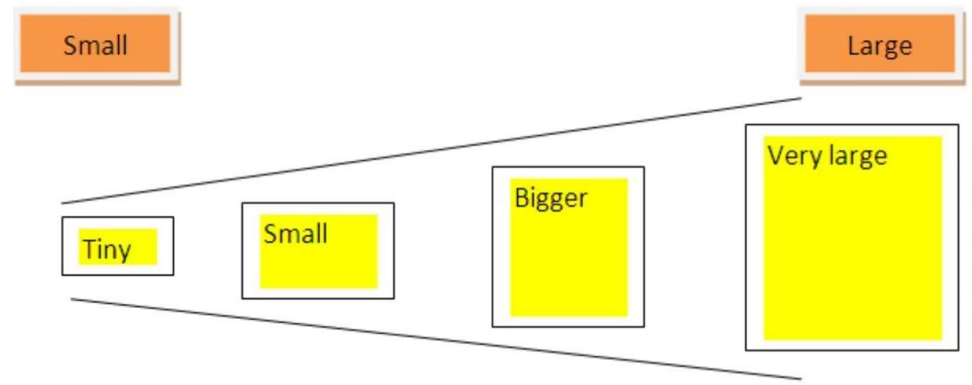
Estimate with T-Shirt Size

- Relative estimation technique, usually applied to user stories
- In planning poker, rather than using a number, Items are classified into t-shirt sizes, ex: XS, S, M, L, XL.



©2011-2023 João Cunha, DEIS-ISEC

Affinity Estimating



©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

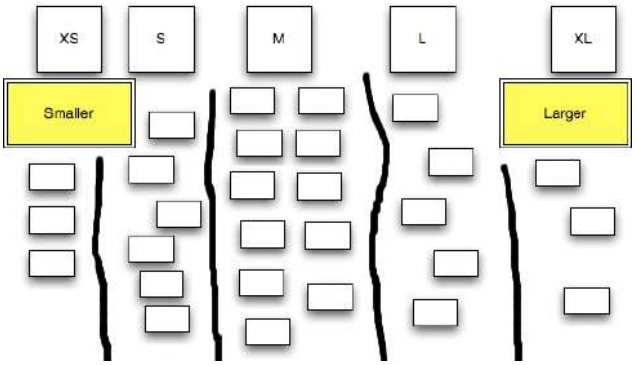
How to estimate – Affinity Estimating

1. Place two posts on the Wall (small and large).
2. Team members will be asked to come to the Wall with a subset of the product backlog.
3. Team members will size each item relatively to the others on the wall.
   - Silent step.
4. Product owner/Stackeholders present only to provide clarity on some issue.
5. Place all the items on the Wall.
6. After that, all members would look to the Wall and validate if they agree on the estimation.
7. They may change the place of the user story after some debate.
8. Estimation ends when no more user stories are changed in the Wall.

©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

How to estimate – Affinity Estimating

9. Create columns
   - Relative buckets.
   - T-shirt or fibonacci sequence



©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Prioritisation with MoSCoW

- In fixed-time projects it is vital to understand the relative importance of the work to be done
- MoSCoW helps to understand and manage priorities, mainly in User Stories

- **M**ust Have
  - there is no point in implementing a solution that does not meet this requirement
- **S**hould Have
  - important but not vital, although painful to leave out, the solution is still viable
- **C**ould Have
  - wanted or desirable but less important, has less impact than “should have” if left out
- **W**on’t Have this time
  - not be delivered for now, maybe later; helps keeping the focus at this timeframe on the more important requirements

©2011-2023 João Cunha, DEIS-ISEC



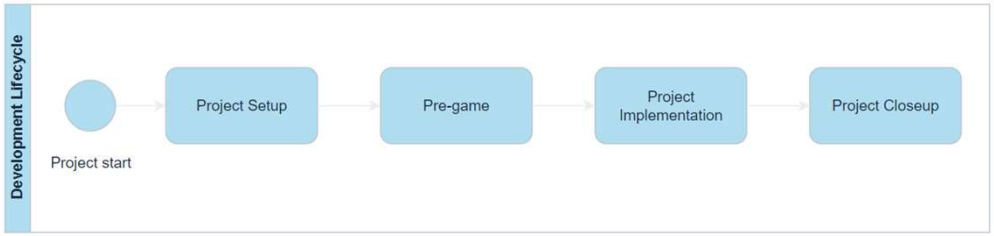
Further reading

- Andrew Stellman and Jennifer Greene, *Applied Software Project Management*, O'Reilly, 2006. [Section 3]
- Whitney Vige, What are story points? Six easy steps to estimate work in Agile, December 3rd, 2022 (https://asana.com/resources/story-points)
- Mountain Goat Software website, *Planning Poker*, https://www.mountaingoatsoftware.com/agile/planning-poker
- Many online tools (ex: https://planningpokeronline.com/)

Pre-game

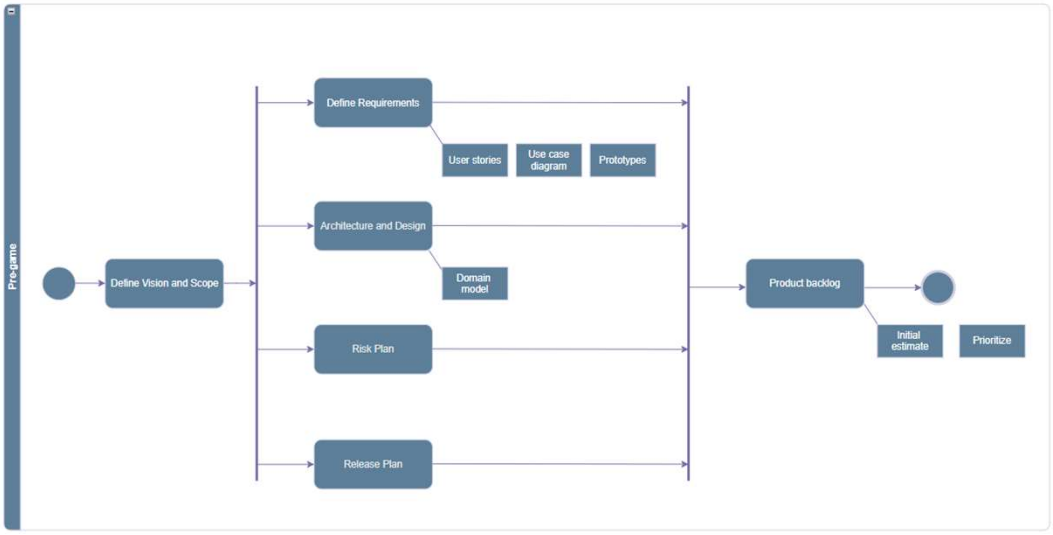
©2011-2023 João Cunha, DEIS-ISEC

Development Lifecycle



©2011-2023 João Cunha, DEIS-ISEC

Pre-Game (or Sprint 0)



©2011-2023 João Cunha, DEIS-ISEC

README.md document

1. Team
2. Vision and Scope
3. Requirements
   1. Use case diagram
   2. Mockups
   3. User Stories (links to GitLab issues)
4. DoD (Definition of Done)
5. Architecture and Design
6. Risk Plan
7. Pre-Game
8. Release Plan
9. Increments
   1. Sprint 1
   2. …

©2011-2023 João Cunha, DEIS-ISEC

Team

- Team members:
  - Name1, student number, GitLab email
  - Name2, student number, GitLab email
  - Name3, student number, GitLab email
  - …

©2011-2023 João Cunha, DEIS-ISEC

Vision and Scope

1 Problem Statement
- 1.1 Project background
- 1.2 Stakeholders
- 1.3 Users

2 Vision & Scope of the Solution
- 2.1 Vision statement
- 2.2 List of features
- 2.3 Features that will not be developed
- 2.4 Assumptions

©2011-2023 João Cunha, DEIS-ISEC

DoD (Definition of Done)

- It is a collection of criteria that must be completed for a User Story to be considered “done.”
- Example:
  1. All tasks done
     1. CI – built, tested (JUnit), reviewed (SonarCloud)
     2. Merge request to **qa** (code review)
  2. Acceptance tests passed
  3. Accepted by the client
  4. Code merged to **main**

©2011-2023 João Cunha, DEIS-ISEC

Risk Plan

- Threshold of Success:
  1. By the 2nd release date, 30% of all the User Stories from the Product Backlog are closed.
  2. *Goal2 …*
- Risk List
  - RSK2 – PxI: 4x5=20; The team has no experience in software estimation, **therefore** there might cause underestimation, delaying the project in an uncontrolled way
  - *RSKn – PxI: axb=c; Risk statement*
  - …
- Mitigation Actions (threats>=20)
  - RSK2 – MS; At the middle of each sprint, compare the progress of the project with the estimation and, if necessary, review the sprint plan.
  - *RSKn – CP/AS/MS; Contingency Plan (CP), Avoidance Strategy (AS) or Minimization Strategy (MS)*
  - …

©2011-2023 João Cunha, DEIS-ISEC

Pre-Game

- Sprint 0 Plan
  - Goal: description
  - Dates: from 10-13/Oct to 24-27/Oct, 2 weeks
  - Sprint 0 Backlog:
    - Task1 – Write Team
    - Task2 – Write V&S
    - Task3 – Write Requirements
    - Task4 – Write DoD
    - Task5 – Write Architecture&Design
    - Task6 – Write Risk Plan
    - Task7 – Write Pre-Game
    - Task8 – Write Release Plan
    - Task9 – Write Product Increments
    - Task10 – Create Product Board
    - Task11 – Create Sprint 0 Board
    - Task12 – Write US in PB, estimate (SML), prioritize (MoSCoW), sort
    - Task13 – Create repository with "GPS Git" Workflow

©2011-2023 João Cunha, DEIS-ISEC

Release Plan

- Release 1
  - Goal: MVP – description
  - Date:
    - [teams 0] 21-24/Nov
    - [teams1] 28-30/Nov
  - Release: V1.0
- Release 2
  - Goal: Final release – description
  - Date:
    - [teams 0+1] 12-15/Dec
  - Release: V2.0

©2011-2023 João Cunha, DEIS-ISEC

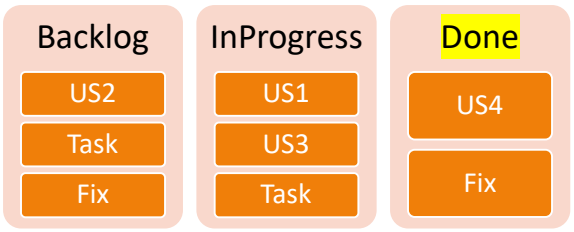
Increments

- Sprint 1
  - Sprint Plan
    - Goal: *what's the goal for this sprint*
    - Dates: from *24-27/Oct* to *7-10/Nov | 14-17/Nov, 2 | 3* weeks
    - Roles:
      - PO: *name*
      - SM: *name*
    - To do:
      - *(list of US or Tasks from the PB, with links)*
      - US1: *As … I want … so that …*
      - Task1: *Some task*
    - Story Points: *2S+3M+3X+2H*
    - Analysis: *short analysis of the planning meeting*
  - Sprint Review
    - Analysis: *what was not done or what was added (Link to US or Task from the PB)*
    - Story Points: *2S+1M+2X+2H*
    - Version: 0.1
    - Client analysis: *client feedback*
    - Conclusions: *what to add/review*
  - Sprint Retrospective
    - What went well:
    - What went not well:
    - What to improve:
- Sprint 2
  - …

©2011-2023 João Cunha, DEIS-ISEC

GPS Product Backlog (in GitLab)

- Product backlog items (or cards) represented as **issues**, in an **issue board**
  - Note: When creating the "Issue Board" in GitLab, remove default "Open" and "Closed" lists
- **Issues board** configuration:
  - **Backlog** (where the product backlog cards are placed)
    - US[X] – User Story number X
  - **InProgress** (when the corresponding cards are copied to the Sprint Backlog)
    - When a US[X] is InProgress, the corresponding AT[X] and Tasks are created in the Sprint Backlog
  - ~~Closed~~ **Done** (when the corresponding cards are ~~Closed~~ Done in the Sprint Backlog)
    - When an AT[X] is ~~Closed~~ Done in the Sprint Backlog, the US[X] is also ~~Closed~~ Done



©2011-2023 João Cunha, DEIS-ISEC

Labels for the Product Backlog

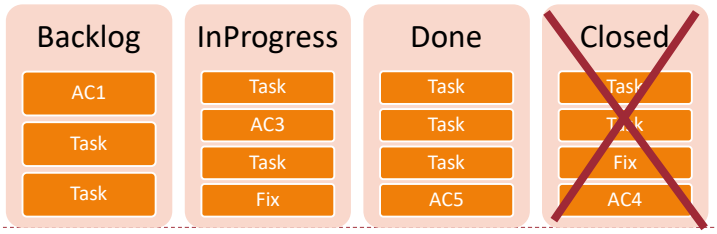
List labels	Card Types	Size Estimation (for US cards only)	Priority (for US cards only)
Product Backlog BacklogPB	User Story <number> US1	Large L	Must have Must
In Progress InProgressPB	Configuration task conf	Medium M	Should have Should
Done DonePB	Documentation task doc	Small S	Can have Can
	Management task man		Won't have Won't
	Any fix fix		

- GPS Product Backlog (issue board) has 3 lists (see **List Labels**)
- The cards (issues) are:
  - US (numbered)
  - 3 "task" types
  - 1 "fix", of any type
- The US cards may have:
  - One of three size estimations
  - One of four priorities

©2011-2023 João Cunha, DEIS-ISEC

GPS Sprint<x> Backlog (in GitLab)

- Sprint backlog cards represented as **issues**, in an **issue board**
- Each Sprint has its own backlog (issue board)
  - Note: When creating the "Issue Board" in GitLab, remove default "Open" and "Closed" lists
- **Issues board** configuration:
  - **Backlog** (where the sprint backlog cards are placed)
    - Each User Story will have its corresponding Acceptance Criteria card (e.g. AC1)
  - **InProgress** (when the cards are being developed; if code, in **dev** branch)
  - **Done** (when the cards are being validated; or the code is in **qa** branch)
  - ~~**Closed** (when the cards are "done"; or the if code is in **main** branch)~~



©2011-2023 João Cunha, DEIS-ISEC

Labels for the Sprint Backlog

List labels	Card Types	Related US (for USTask and AC cards only)
Sprint Backlog BacklogS1	US Task USTask	User Story <number> US1
In Progress InProgressS1	Acceptance Criteria AC	
Done DoneS1	Configuration task conf	
~~Closed~~ ~~ClosedS1~~	Documentation task doc	
	Management task man	
	Any fix fix	

- GPS Sprint Backlog has ~~4~~ 3 lists
- The cards (issues) are:
  - **USTask**
  - **AC**
  - 3 "**task**" types
  - 1 "**fix**", of any type
- All card types should have an Estimated time
- The USTask and AC cards should have:
  - A related User Story

©2011-2023 João Cunha, DEIS-ISEC

187

Backlog card types

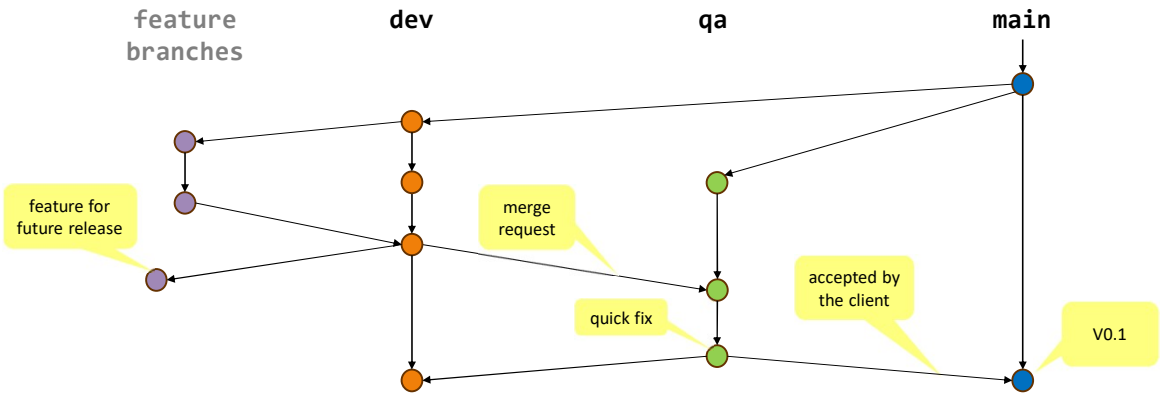
card	Title	Description	Estimate	Priority	Tasks	Labels	Boards
User Story	US[X]: As a …, I want … so that…	Short description; Link to acceptance tests	S, M, L story points	MoSCoW	None	US[X]	PB
User Story Task	USTask: Short description	Full description of the task	0, 1, 2, 3, 5 or 8 hours		necessary subtasks	US[X] & USTask	SB
Task – documentation, configuration, management	Task: Short description	Full description	0, 1, 2, 3, 5 or 8 hours		necessary subtasks	doc | conf | man	PB or SB
Fix – code, documentation, configuration	Fix: Short description	Full description	0, 1, 2, 3, 5 or 8 hours		necessary subtasks	Fix & (US[X] | doc | conf)	SB or PB
Acceptance Criteria/Test	AC: Acceptance criteria for US[X]	All acceptance criteria/tests for the US[X]	0, 1, 2, 3, 5 or 8 hours		one per acceptance test plus DoD criteria	US[X] & AC	SB

©2011-2023 João Cunha, DEIS-ISEC

188

GPS Git workflow

- Branches



©2011-2023 João Cunha, DEIS-ISEC

Planning

©2011-2023 João Cunha, DEIS-ISEC

Release Planning

- High-level document that outlines the scope, timeline, and goals of the releases of a product. The first is usually an MVP (Minimum Viable Product)
- Not part of Scrum framework, but is often used
- It's necessary to have:
  - What the Client needs/wants, and when
  - Risk plan, with mitigation tasks
  - Most architectural relevant parts
  - Release time
  - A list of User Stories (in the **Product Backlog**):
    - Priotitized (ex: MoSCoW)
    - Estimated (in Story Points, ex: XS, S, M, L, XL)
  - Team capacity (hours/week * team members * duration in weeks)

©2011-2023 João Cunha, DEIS-ISEC

Release Planning

- How to plan:
  - Order the US, by putting on top:
    - Those with the highest business value (chosen by the client)
    - The most architectural relevant
    - Those with the highest risk
    - Those with the highest priority
  - Select some related US from the top of the list. Define a goal, that represents an MVP
  - Have a rough idea of the effort to develop the MVP: convert story points into hours (ex: S:0-2; M:3-5; L:6-10; XL:11-18)
  - Compare with the Team Capacity for the release date. Adjust as necessary.
    - Team Capacity = week hours per person * number of weeks * number of persons
  - Write the Release goal.
- Continue for the next releases

©2011-2023 João Cunha, DEIS-ISEC

Sprint Planning

1. Based on the next Release plan and top of the Product Backlog, select the related User Stories for the next sprint
2. Estimate the Team Capacity for the sprint. Check if the US can be finished.
3. Extract the Tasks and Acceptance Criteria for each US, and add them to the **Sprint Backlog**
4. Estimate the effort for each Task and AC (in hours)
5. Verify if the Tasks and AC can be done in the sprint. Review if necessary
6. Write the Sprint Goal

©2011-2023 João Cunha, DEIS-ISEC

197

Monitoring and Control

©2011-2023 João Cunha, DEIS-ISEC

198

Managing Projects

- Planning
  - Define tasks, estimate, schedule, assign resources
- Monitoring
  - Keep track of the progress and evaluate performance (compare actual status with the plan)
- Control
  - Change scope, schedule, reassign resources, reestimate

©2011-2023 João Cunha, DEIS-ISEC

What to look at?

- Is the project on schedule?
  - How much time has elapsed?
- Is the project on budget?
  - How much did it cost so far?
- Is the work being accomplished as planned?
  - How much of the planned work has been realized?

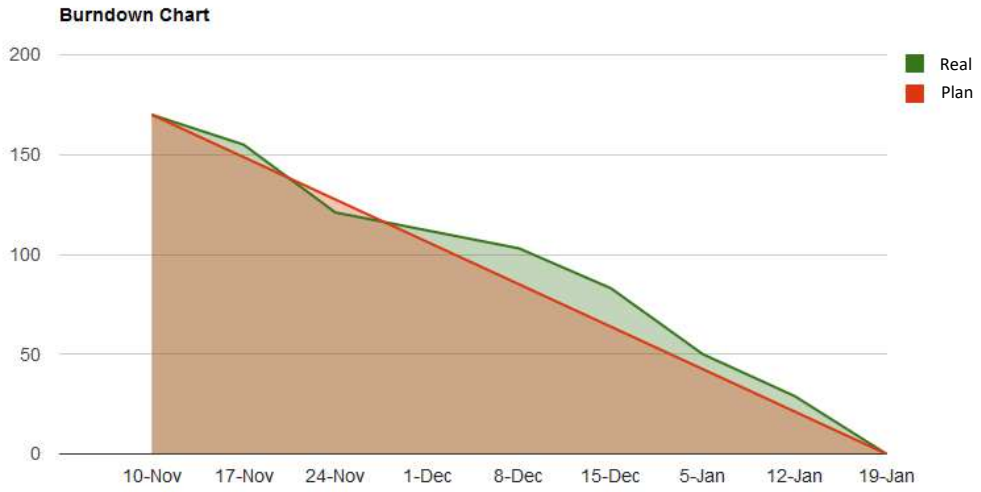
©2011-2023 João Cunha, DEIS-ISEC

1. Burndown chart

- A **burndown chart** is a graphical representation of work left to do versus time.
- Every task has an estimated work effort
  - Every day the remaining work for each task is updated
- It is useful for predicting when all the work will be completed.
- However, the quality of the burndown chart depends on the accuracy of the tasks estimated effort
- Also, the burndown chart does not deal with the actual work

©2011-2023 João Cunha, DEIS-ISEC

Burndown chart example



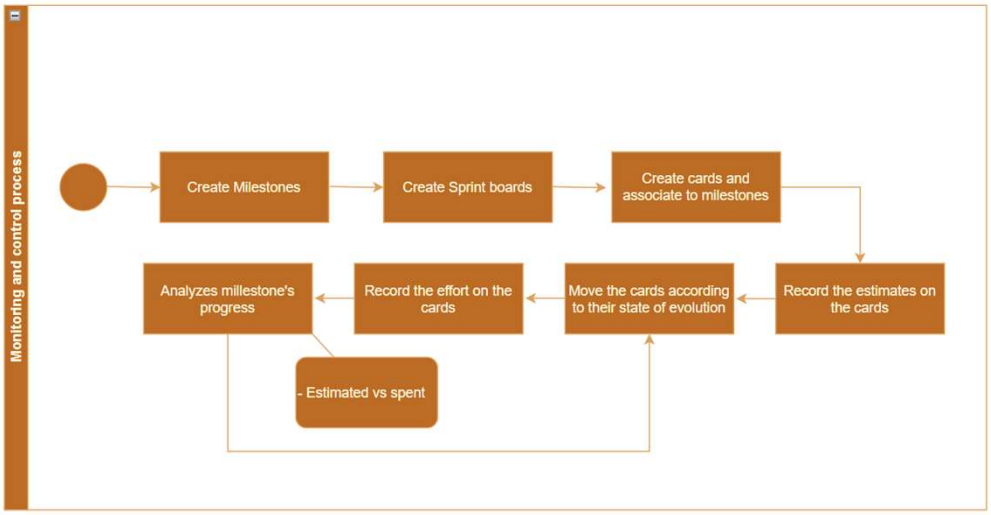
©2011-2023 João Cunha, DEIS-ISEC

How burndown chart works

- The work is planned until the team capacity
  - # of team members × # of days in the iteration × # of hours per day (conservative – e.g. 6 hours)
- The ideal daily burndown is:
  - team capacity / # of days in the iteration
- Every day, the work left to be done for the project iteration is calculated:
  - if a task is completed, its remaining effort is zero
  - if a task has been started, but not completed, either:
    - its remaining effort is updated
    - its remaining effort is left unchanged (all or nothing)
- If a new task is added, its estimated effort is added to the chart

©2011-2023 João Cunha, DEIS-ISEC

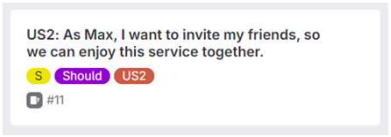
Using GitLab



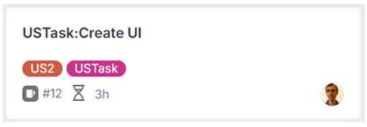
©2011-2023 João Cunha, DEIS-ISEC

Write Estimation and Real Effort

- For User Stories

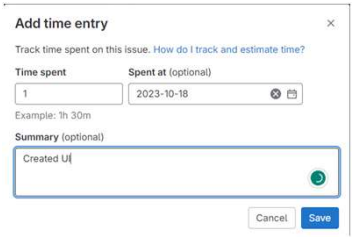


- For Tasks (issues)



  - Add estimate
  - Add time entry



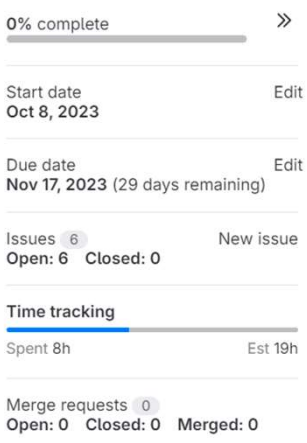


©2011-2023 João Cunha, DEIS-ISEC

205

Tracking Sprint

1. Create a Milestone for each sprint
2. Assign the Sprint Tasks (Issues) + AC to the Milestone
3. When task starts, assign to developer
4. When some work is done, add time entry
5. When all tasks from a US is done, start AC
6. When AC is done, create a merge request
7. When accepted, move Tasks+AC to Done
8. Track project:
   1. Time tracking
   2. Merge requests
9. Sprint review:
   1. When US accepted by the Client, Close UC+Tasks+AC



©2011-2023 João Cunha, DEIS-ISEC

206

Daily Scrum, Sprint Review, Sprint Retrospective

©2011-2023 João Cunha, DEIS-ISEC

Daily Scrum (2020 version)

- Also known as Daily Standup Meeting
- Purpose: inspect progress toward the Sprint Goal
  - May adapt the Sprint Backlog, adjusting the upcoming planned work.
- Short meeting with Developers
  - Daily, 15 minutes, same place
- Structure and techniques are selected by the Developers
  - Focus on the progress toward the Sprint Goal
  - Produces an actionable plan for the next day of work
- Daily Scrums:
  - Improve communications
  - Identify impediments
  - Promote quick decision-making
  - Eliminate the need for other meetings

©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Sprint Review (2020 version)

- Purpose: to inspect the outcome of the Sprint and determine future adaptations.
- Meeting With Scrum Team and Stakeholders:
  - At the end of the sprint
  - The Scrum Team presents the results of their work
  - Attendees discuss progress toward the Product Goal
  - Attendees review what was accomplished and what has changed
  - Attendees collaborate on what to do next
  - The Product Backlog may be adjusted to meet new opportunities
  - It is a working session, not just a presentation.
- The Sprint Review is timeboxed, max. of 4 hours for a one-month Sprint

©2011-2023 João Cunha, DEIS-ISEC

Sprint Retrospective (2020 version)

- Purpose: to plan ways to increase quality and effectiveness.
- Meeting with the Scrum Team:
  - Inspects how the last Sprint went with regards to individuals, interactions, processes, tools, and their Definition of Done
  - The Scrum Team discusses what went well, what problems it encountered, and how those problems were (or were not) solved.
  - The Scrum Team identifies the most helpful changes to improve its effectiveness
- The Sprint Retrospective concludes the Sprint. It is timeboxed, max. 3 hours for a one-month Sprint.

©2011-2023 João Cunha, DEIS-ISEC

Further reading

- Ken Schwaber & Jeff Sutherland, The Scrum Guide - The Definitive Guide to Scrum: The Rules of the Game, November 2020 (https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf; https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Portuguese-European.pdf)

©2011-2023 João Cunha, DEIS-ISEC